

Object Oriented Analysis & Design (OOAD)(CSE307)

Unit- I

- ❖ Introduction,
- ❖ Class Modeling,
- ❖ Advanced Class Modeling

Terms related to Our Topic

- Problem Statement
- Problem Solving
- Computational Solution
 - Structured approach
 - Earlier, information systems used to be defined mainly by their **functionality: data and functions were kept separate** and connected together by means of input and output relations.
 - Object Oriented approach
 - it focuses on objects that represent abstract or concrete things of the real world.
 - Objects defined by their character along with the properties (attributes)
 - The behavior of these objects is described by methods.
 - It follows bottom-up approach because in OOP, we first identify the smallest parts of the program i.e. the objects. We then combine these objects to develop the complete program.

Structured Vs. Object-Oriented

Structured Analysis

The main focus is on process and procedures of system.

It uses System Development Life Cycle (SDLC) methodology for different purposes like planning, analyzing, designing, implementing, and supporting an information system.

It is suitable for well-defined projects with stable user requirements.

Risk while using this analysis technique is high and reusability is also low.

Structuring requirements include DFDs (Data Flow Diagram), Structured English, ER (Entity Relationship) diagram, CFD (Control Flow Diagram), Data Dictionary, Decision table/tree, State transition diagram.

This technique is old and is not preferred usually.

Object-Oriented Analysis

The main focus is on data structure and real-world objects that are important.

It uses Incremental or Iterative methodology to refine and extend our design.

It is suitable for large projects with changing user requirements.

Risk while using this analysis technique is low and reusability is also high.

Requirement engineering includes Use case model (find Use cases, Flow of events, Activity Diagram), the Object model (find Classes and class relations, Object interaction, Object to ER mapping), Statechart Diagram, and deployment diagram.

This technique is new and is mostly preferred.

OOAD - Objectives

- “Object Oriented” is a way of thinking abstractly about a problem using real world concepts rather than computer concepts
- Graphical notation helps to visualize the problem.

OOAD - Objectives

- ✓ *Object Oriented approach* to s/w development based on modeling objects from the real world, then Using the model to build a language-independent design.
- ✓ OO Modelling, design & language-independent graphical notations promotes better understanding of
 - **Requirements, cleaner design & maintainable system**
 - **Analyze problem requirements**
 - **Design solutions to the problem**
 - **Implement solution in a programming language/DB**
 - **Same concepts & notation to be used throughout s/w development process**
 - **The developer need not to translate into new notation at each development stage.**

Analysis ? Design ? Implementation

Object Oriented Approach, Development & Methodology

- 1 Object Oriented Approach – discrete objects - abstraction
- 2 Object Oriented Development - software based on abstractions
- 3 Object Oriented Methodology – process for OO development

Object Oriented Approach

Object Orientation

- OO means that we **organize software as a collection of discrete objects that incorporate both data structure and behavior** (tightly coupled).
- Whereas the previous programming approaches are only loosely coupled.
- Characteristics required by OO approach***
 - Identity
 - Classification
 - Inheritance
 - Polymorphism

Identity

- Data is quantized into discrete, distinguishable entities called – object (eg: first paragraph in this chapter, my laptop, white queen in a chess game)
 - Object has its own inherent identity.
 - two objects are distinct even if their attributes are same
 - are instances of a class
 - interact through method
 - in programming language, object has a unique handle by which it can be referenced (address, array index address,

Chapter 1 / Introduction

variable name	address
aCredit	10000007
aDebit	13537163
anAccount	56826358
aSavingsAccount	45205128

a symbol table



a binary tree



a monitor



Mike's bicycle



Brian's bicycle



a white rook

Figure 1.1 Objects. Objects lie at the heart of object-oriented technology.

Classification

- Objects with the same data structure (attributes) and behavior are grouped into a **class** (eg: **Paragraph, Monitor, and ChessPiece**).
- A class is an abstraction/template that describes properties important to an application and ignores the rest.
- Each class describes infinite set of individual objects.
- Each object is said to be an instance of class.
- Each instance of class has its own value for each attributes but shares the attribute names and operations with other instances of the class.

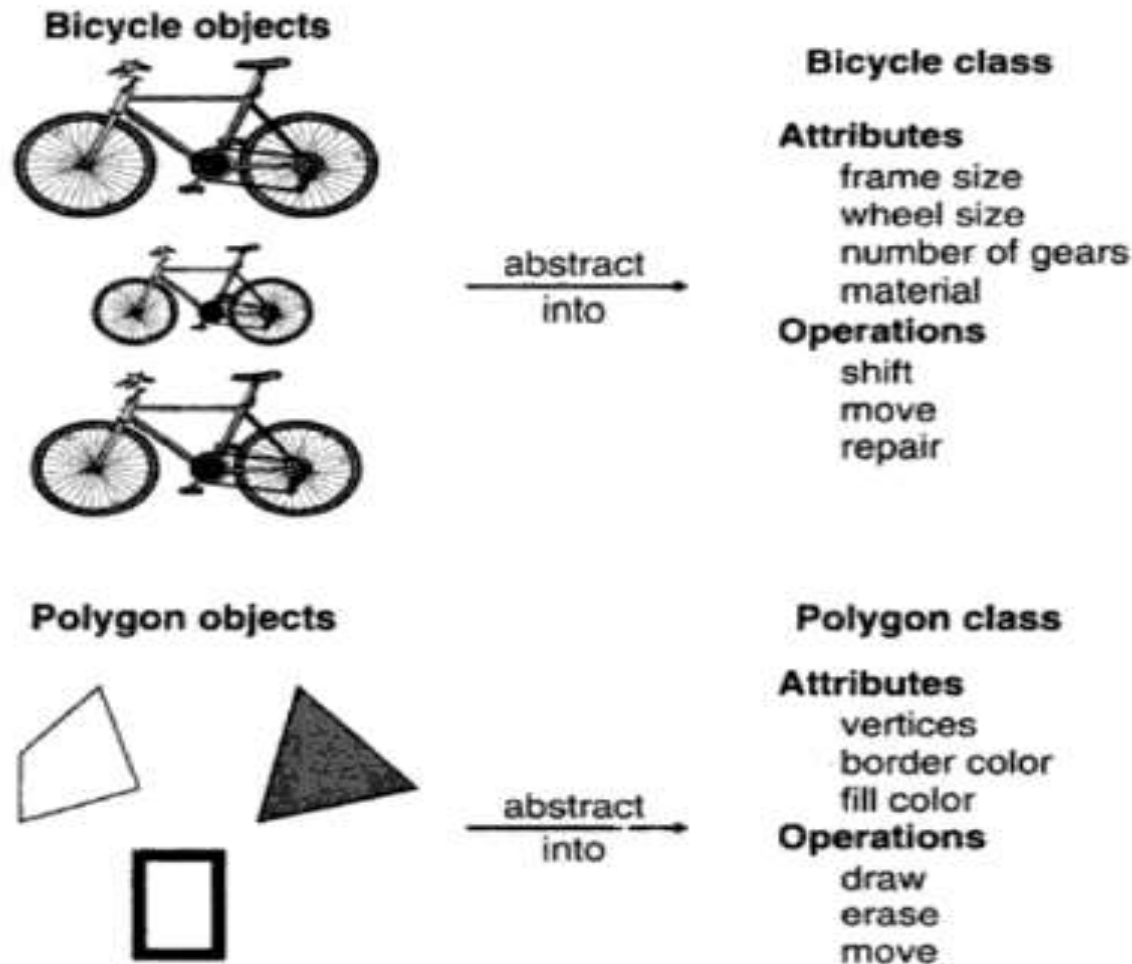


Figure 1.2 Objects and classes. Each class describes a possibly infinite set of individual objects.

Inheritance

- **Sharing of attributes and operations among classes** based on a hierarchical relationship.
- Super class – it has general information.
- Sub class – it has refined / elaborate information which inherits all the features of its super class and adds its own unique features.
 - Each subclass inherits all the features of its super class & adds its own unique features.
 - **Ex:- ScrollingWindow and FixedWindow are subclasses of Window.**
- Advantage – inheritance greatly **reduce repetition within designs and programs**

Polymorphism

- Same operation may behave differently on different classes
- **Example:** move operation may behave differently for pawn and queen in a chess game.
- **Operation** – is a procedure or transformation that an object performs – (eg:-RightJustify, display, move)
- **Method** - A specific **implementation of an operation** by a certain class is called a method
- Object knows how to perform its own operations

Object Oriented Development

- It is a new way of thinking about software based on abstractions that exist in the real world as well as in the program.
- Development refers to s/w life cycle.
 - Analysis, Design and Implementation
- OO development is a conceptual process, independent of programming language until final stage.
- **Modelling** concepts **not implementation**
 - Important for what to do rather than how to do

Play match

- | | |
|-------------------------|---|
| 1. Identity | a) your mobile |
| 2. Object | b) distinguish objects of same class |
| 3. Student (regno,name) | c) diagram |
| 4. Class | d) data-structure |
| 5. UML | e) abstraction |
| 6. Employee | f) visualize the system design |
| 7. behaviour | g) eid,ename,esalary,price |
| 8. OO modelling | h) id,name,salary,applyCL() |
| 9. Super-class | i) issueBook, bookNo, returnBook, |
| 10. Inheritance | j) searchBook, addBook, cancelOrder |
| | k) construction of objects based on their stored values |
| | l) Generalized version |
| | m) Specialized version |
| | n) re-usability |

Answer:

Object Oriented Methodology

- **Object Oriented Methodology** is a process for object **oriented development** and a graphical notation for representing **object oriented concepts**.
- **Process:** consists of **building a model** of an application; and then adding details to it during design (this is called OMT object modeling technique).
- Methodology has the following stages:
 - **System Conception**
 - s/w development begins with formulating tentative requirements
 - **Analysis**
 - Analyst works with requester and scrutinizes the requirements, specifies “what the system must to do” ;
 - The analysis model should not contain implementation decisions.
 - Analysis model has **domain model** and **application model**
 - **Domain Model:** description of real-world objects reflected within the system
 - **Application Model:** description of the parts of the application system ; visible to user.

Example: Stockbroker Application

Stock, bond, trade and commission □ **Domain objects**

Execution of trades, present the results □ **Application objects**

Example: Hybrid Examination

question_paper, duration, marks □ **Domain objects**

Conducting exam, evaluate answer scripts, publish results □ **Application objects**

Continued..

- System design

- Development team work out for *the system architecture* , Establish policies and decide performance characteristics , resource allocations.
- Example: System designer decides for a **stock display work station**, to display proper communication messages/error messages.

- Class design

- Class designer adds details to **Analysis model accordance with System design**
- Class designer elaborates **domain** and **application** objects using **OO concepts & notation**.
- Main focus is **data structures** and **algorithms needed to implement each class**

- Implementation

- Implementers translate the classes and relationships developed during class design into a particular programming language, database or h/w
- Should follow s/w engineering practice for traceability.

Continued..

Note:

1. We are **not suggesting waterfall development** process (requirement, analyzing, designing and implement)
2. Suggest iterative process – **developing part of the system through several stages** and then adding capability
3. Some classes are not part of analysis but are introduced during design/implementation.
4. We don't consider Testing as a distinct step, but it must be part of QC occurs throughout life cycle.

OO Themes

- **Abstraction**
- **Encapsulation**
- **Combining data and behavior**
- **Sharing**
- **Emphasis on the essence of an object**
- **Synergy**

Abstraction

- Focus on **essential aspects** of an application while ignoring details. This means focusing on what an object is and does, before deciding how to implement it.
- Use – avoiding premature commitments to details
- **Abstraction in Real World** : Making coffee with a coffee machine – no need to know is how the coffee machine is working internally
- **Abstraction in OOP**: hides the internal implementation details – just need to know which methods of the object for calling, parameters needed.

Encapsulation

- *Encapsulation – (also information hiding)*
- Separates **the external aspects of an object** that are accessible to other objects, from the internal implementation details that are hidden from other objects.
- Use – **prevents portions of a program becoming so interdependent.**
- *Encapsulation in programming:*
 - wrapping up of data under a single unit.
 - It binds code and the data together
 - it is a protective shield that prevents the data from being accessed by the code outside this shield

Combining Data and Behavior

- With OO program each **object implicitly decides which procedure to use, based on its class**. so the caller of an operation need not consider how many implementations exist. (**polymorphism**)
- Advantage – maintenance is easier, because the calling code need not be modified when a new class is added.
- **Student** ? objUgStudent - regno, name,
 - Operations: search();
- **Library** ? objdeptLibrary
- Operation/behaviour : search();

Sharing

- OO techniques promote sharing at different levels.
- **Inheritance** of both data structure and behavior lets subclasses share common code.
- Reusing designs and code for other applications also.

Emphasis on the essence of an object

- OO development places **greater emphasis** on **data structure** (because object structure are more stable in the long run) and a **lesser emphasis** on **procedure structure**.
- Use – adds the concept of class-dependent behavior.

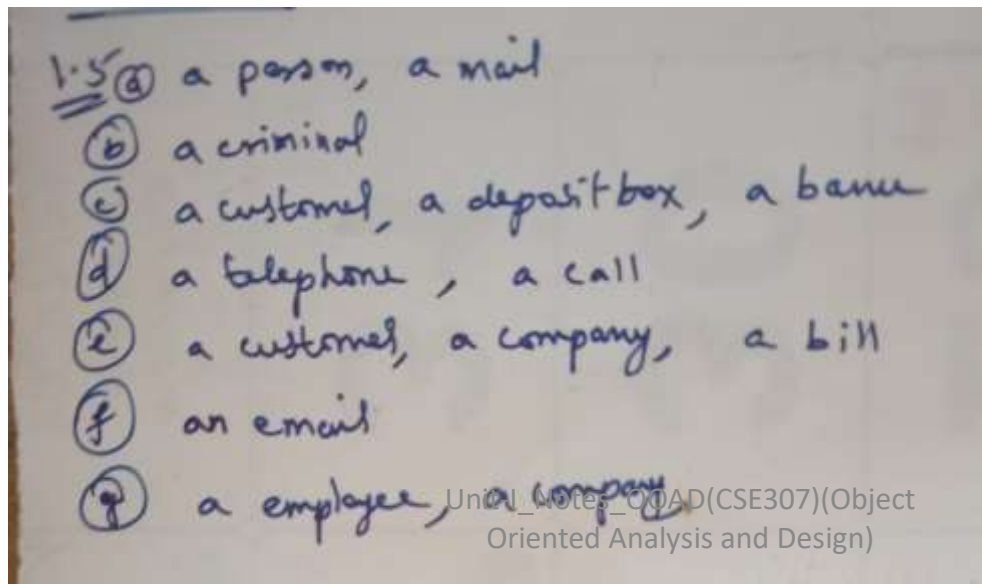
Synergy

- By combining the characteristics of OO approach synergistically the resulting system tends to be cleaner, more-general and more robust

Exercise

(5) All objects have identity and are distinguishable. However, for large collections of objects, it may not be a trivial matter to devise a scheme to distinguish them. Furthermore, a scheme may depend on the purpose of the distinction. For each of the following collections of objects, describe how they could be distinguished.

- All persons in the world for the purpose of sending mail
- All persons in the world for the purpose of criminal investigations
- All customers with safe deposit boxes in a given bank
- All telephones in the world for making telephone calls
- All customers of a telephone company for billing purposes
- All electronic mail addresses throughout the world
- All employees of a company to restrict access for security reasons



- (4) Prepare a list of classes that you would expect each of the following systems to handle.
- A program for laying out a newspaper
 - A program to compute and store bowling scores
 - A telephone voice mail system with delivery options, message forwarding, and group lists
 - A controller for a video cassette recorder
 - A catalog store order entry system

1.6
(a) Newspaper (class) ; a newspaper. (object)
- attributes: Date, Pages, Heading News, ...
- operations: designHeading(), editContent()

(b) ~~Bowling~~ Scores → class bowlingScore → object
attributes: score
operations: computeScore(), storeScore()

(c) Telephone (or) VMS
attributes: delivery option, message, list
operations: forwardMessage(), group list()

Unit I_Notes_OOAD(CS5307)(Object Oriented Analysis and Design)

Exercise

(6) Classes and operations are listed below. For each class, select the operations that make sense for objects in that class. You may place an operation in multiple classes. Discuss the behavior of each operation.

Classes:

variable-length array — ordered collection of objects, indexed by an integer, whose size can vary at run time

symbol table — a table that maps text keywords into descriptors

set — unordered collection of objects with no duplicates

Operations:

append — add an object to the end of a collection

copy — make a copy of a collection

count — return the number of elements in a collection

delete — remove an element from a collection

index — retrieve an object from a collection at a given position

intersect — determine the common elements of two collections

insert — place an object into a collection at a given position

update — add an element to a collection, writing over whatever is already there

1.7 Chapter: 1

Variable-length Array (class)

- append()
- copy()
- count()
- delete()

Set (class)

- append()
- copy()
- count()
- delete()

index()

intersects()

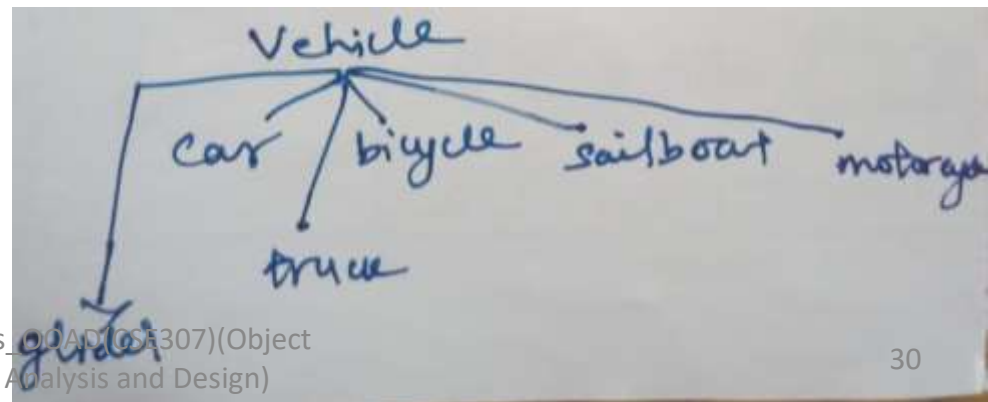
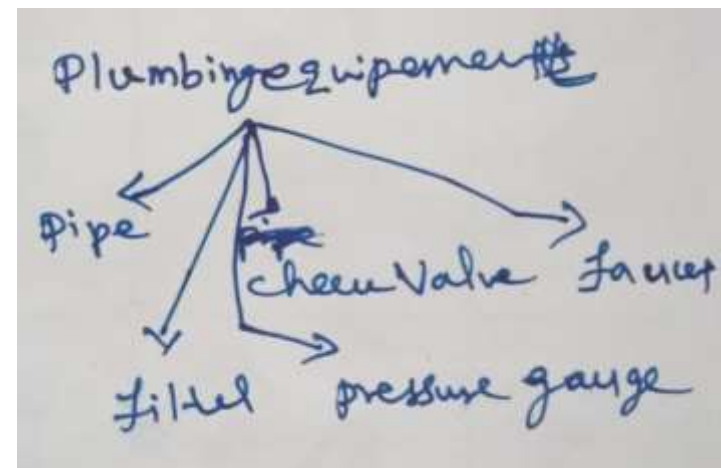
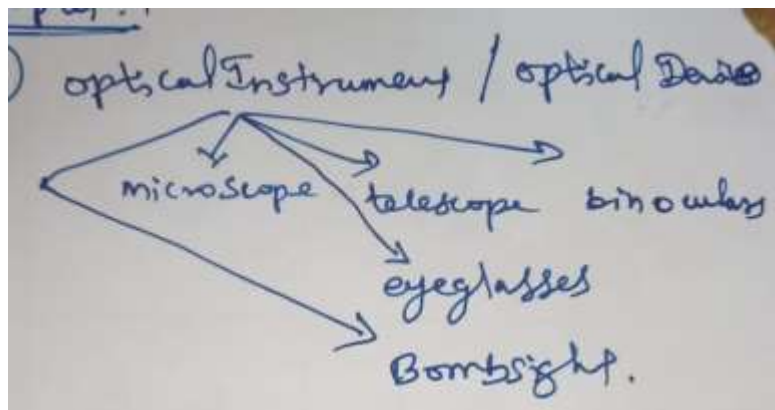
insert()

update()

Exercise

(4) Discuss what the classes in each of the following lists have in common. You may add more classes to each list.

- scanning electron microscope, eyeglasses, telescope, bomb sight, binoculars
- pipe, check valve, faucet, filter, pressure gauge
- bicycle, sailboat, car, truck, airplane, glider, motorcycle, horse
- nail, screw, bolt, rivet
- tent, cave, shed, garage, barn, house, skyscraper



Modeling

What is a model?

A model is an abstraction of something for the purpose of understanding it before building it.

Ex: Building models, flats with environment

Reasons to have a model:

1. Omits nonessential details
2. Easier to manipulate than the original entity
3. Designers build many kinds of models before constructing

Modeling - purpose

Models serve several purposes

1. Testing a physical entity before building it
2. Communication with customers
3. Visualization
4. Reduction of complexity

Three Models

- The three models are separate parts of the description of a complete system but are cross-linked.
 - It is useful to model a system from three related but different viewpoints each capturing important aspects of the system
-
- **Class Model**
 - **State Model**
 - **Interaction Model**

Class Model

1. Class model for the objects in the system and their relationship
 - Describes static structures of objects (attributes and operations)
 - defines context for s/w development.
 - It contains ***class diagram***:
 - ***its a graph whose nodes are classes, arcs are relationships among them.***

State Model

2. State model for the life history of objects

- Describes the aspects of objects that change over time.
- It specifies control with state diagrams.
- concerned with time and the sequencing of operations
- State diagrams express state model.
- **State diagram** – is a graph whose **nodes are states** and whose **arcs are transitions** between states caused by events
- Actions and events in a state diagram become **operations on objects** in the class model.

Interaction Model

3. Interaction model for the interactions between objects
 - Describes how individual objects collaborate to achieve the behavior of the system as a whole.
 - **Use cases**
 - Focuses on functionality of a system
 - **Sequence diagrams**
 - Shows the objects that interact and the time sequence of their interactions.
 - **Activity diagrams**
 - Important processing steps

Class Modeling

Class Modeling

- The purpose of class modeling is to describe objects. It has the following concepts:

1. Objects
2. Classes
3. Values and Attributes
4. Operations
5. Link and Associations
6. Generalization and Inheritance

1. Objects

- An object represents an ***entity , concept , abstraction or thing*** and meaning of the problem at hand
- Objects can be: *Concrete, or Conceptual*
- Objects serve two purposes:
 - Understanding of the real world
 - Provides a practical basis for computer implementation

Examples of objects

- A building



- A form



2. Classes

- A class is a **definition of an object**
- *A class is a set of objects that share a common structure and a common behavior*
- A class is a template (or “blueprint”) for producing objects of a certain type
- Every object is an **instance of a class**, describes a group of objects with **similar properties, common behavior**.
- Examples of classes:
 - Building
 - Form

Classes (cont'd.)

- A class is an *abstraction*:
 1. Shows relevant characteristics
 2. Hides other characteristics
- A class is an *encapsulation*:
 - A set of properties (attributes)
 - A set of operations

Class Diagrams

- It provides a formal graphic notation for modeling objects , classes and their relationships to one another.
- Useful for **modelling and designing actual programs**
- Class Name - ***Singular nouns*** and **noun phrases**
- Concise
- Easy to understand
- Work well in practice
- There are two kinds of models of structure
 - Class diagrams
 - Object diagrams (**Instance diagrams**)

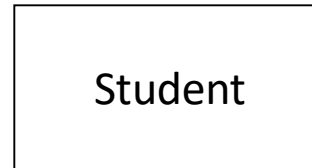
Class Diagram

- It is a schema , pattern, or template for describing many possible instances of data.
- It describes **object classes**.
- it has infinite set of instance diagrams.
- OMT symbol is **rectangle box**

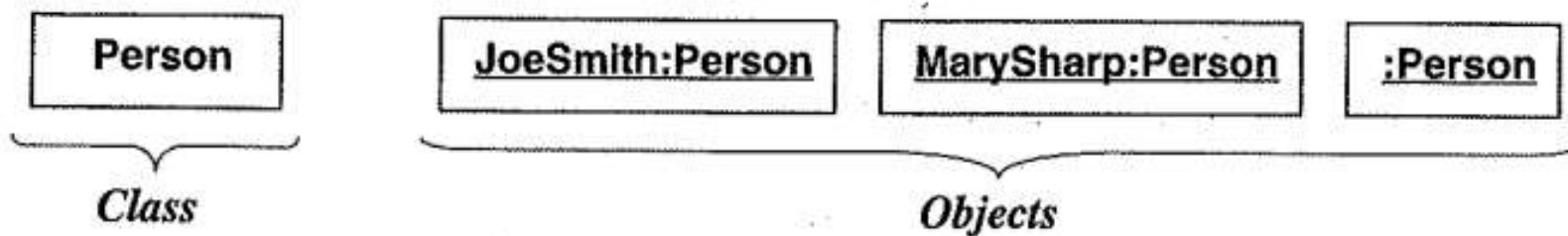
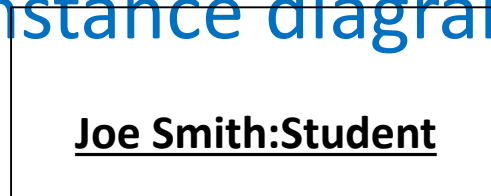
Instance Diagram

- It describes how a particular set of objects relate to each other.
- It describes **object instances**
- It is very useful for documenting test cases and discussing **examples of a complex class diagram**.
- OMT symbol is **also a box but with an object name followed by a colon and the class name**

- Class diagrams



- Instance diagrams



A class and objects. Objects and classes are the focus of class modeling.

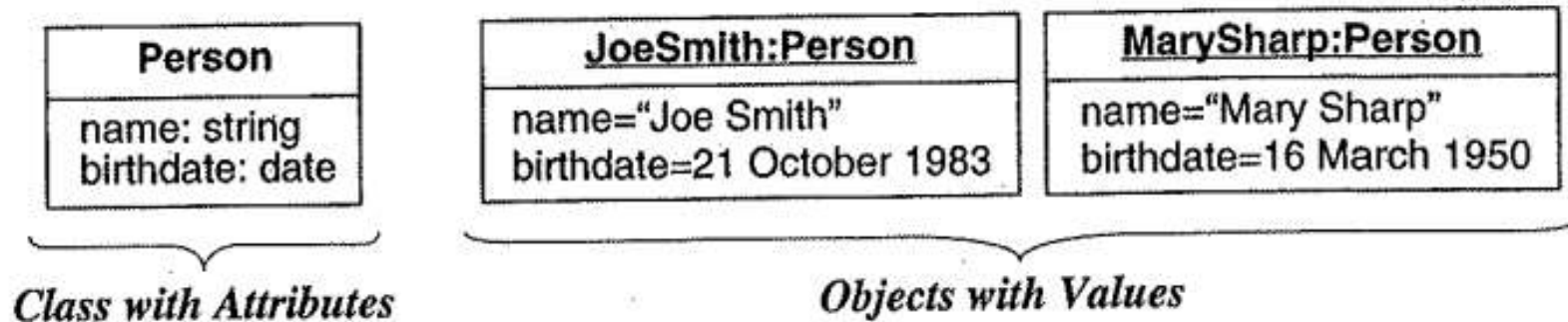
3. Values and Attribute

- A **value** is a piece of data – *get from problem document.*
- An **attribute** is a **named property** of a class – describes a value held by the objects in a class
- Each attribute name is unique within a class
- Attribute name – *adjectives*
- An attribute describes values, not objects
- Attributes are listed in the second part of the class box
- Ex: **Name**, **birthdate** and **weight** are attributes of **Person object**

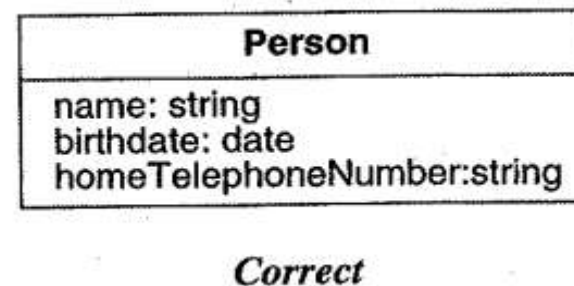
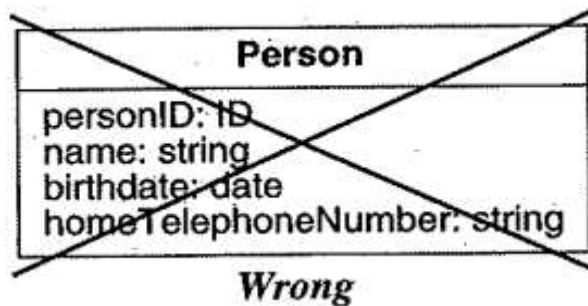
Attribute (Property)

- Each attribute name may be followed by optional details, **such as type and default value**.
- The **type is preceded by colon(:)**, the default value is **preceded by an equal sign(=)**.
- **Need not explicitly list identifiers**, because internal identifiers are purely an implementation convenience and have no meaning in the problem domain
- NOTE: do not confuse internal identifiers (emp-id) with real-world attributes (license no, social security no.)

Attributes-Example



Attributes and values. Attributes elaborate classes.



Object identifiers. Do not list object identifiers; they are implicit in models.

4.Operations

- An operation is a function or procedure that may be applied to or by objects in a class
- All **objects** in a class share the *same operations*
- Same operation may apply to many different classes such an operation is polymorphic
- Operations are listed in the lower third of the class box
- A method is the implementation of an operation for a class
 - when an operation has methods on several classes , then all methods have same **signature** - *The number and types of arguments and the return type (result value)*

Name
 (Identifier)

Variables
 (Static attributes)

Methods
 (Dynamic behaviors)

Student	Circle
name grade	radius color
getName() printGrade()	getRadius() getArea()

SoccerPlayer	Car
name number xLocation yLocation	plateNumber xLocation yLocation speed
run() jump() kickBall()	move() park() accelerate()

Examples of classes

Class Definition

Circle
-radius:double=1.0 -color:String="red"
+getRadius():double +getColor():String +getArea():double

Instances

<u>c1:Circle</u>	<u>c2:Circle</u>	<u>c3:Circle</u>
-radius=2.0 -color="blue"	-radius=2.0 -color="red"	-radius=1.0 -color="red"
+getRadius() +getColor() +getArea()	+getRadius() +getColor() +getArea()	+getRadius() +getColor() +getArea()

Name

Variables

Methods

<u>paul:Student</u>	<u>peter:Student</u>
name="Paul Lee" grade=3.5	name="Peter Tan" grade=3.9
getName() printGrade()	getName() printGrade()

Two instances of the class Student

Summary of Class

Person
name birthdate
changeJob changeAddress

File
fileName sizeInBytes lastUpdate
print

GeometricObject
color position
move (delta : Vector) select (p : Point): Boolean rotate (in angle : float = 0.0)

Operations

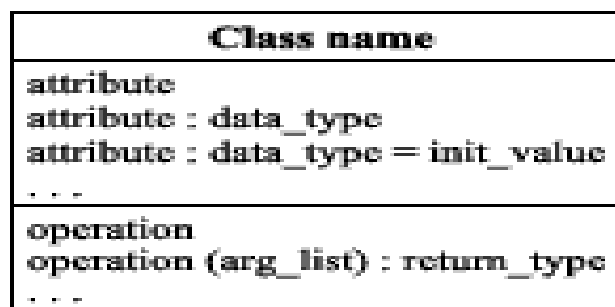
- GeometricObject has move, select and rotate operations
- move** has argument **delta**, which is type **Vector**
- rotate** has argument angle - which is an input of type float with default value 0.0

ClassName
attributeName1 : dataType1 = defaultValue1 attributeName2 : dataType2 = defaultValue2 ...
operationName1 (argumentList1) : resultType1 operationName2 (argumentList2) : resultType2 ...

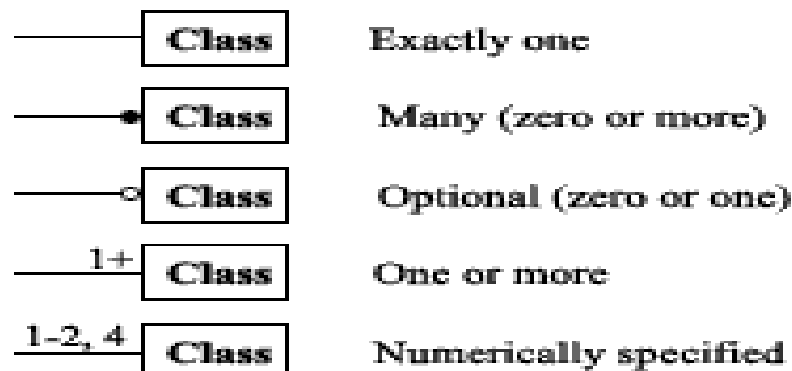
direction argumentName : type = defaultValue

Class:

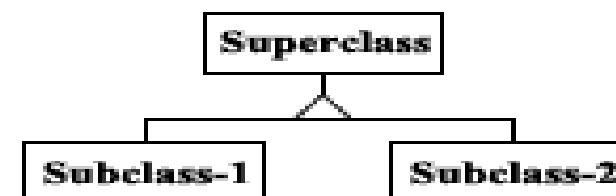
Class Name



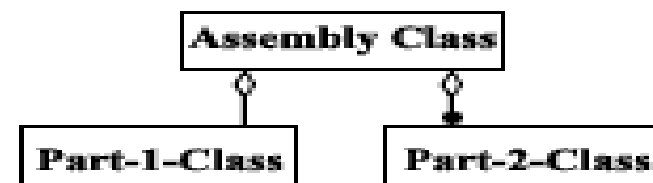
Multiplicity of Associations:



Generalization (inheritance):



Aggregation:



Association:



5. Links and associations

- Links
- After identifying the classes, we should identify the **links** and **associations**, means for **establishing relationships among objects and classes**
- A link is **a physical or conceptual connection** between objects instances
- A link is an instance of an association
- Example: **ram works for ABC company**
- Association
- Association represents **static relationship** shared among the objects of two classes
- An association describes a **group of links with common structure and common semantics**.
- Example : **person works for a company**
- Associations and links often appear as **verbs** in a problem statement.

5. Links and associations

- UML notation for link:
 - a line between objects
 - a line may consist of several line segments.
 - If the link has a name, it is underlined.
 - Association name is optional, **if the model is unambiguous**
 - Association name is mandatory when a model has **multiple associations** among the same classes
 - Eg: - *person **works** for company and
person **owns stock** in company*

Types of Association

- An association can be

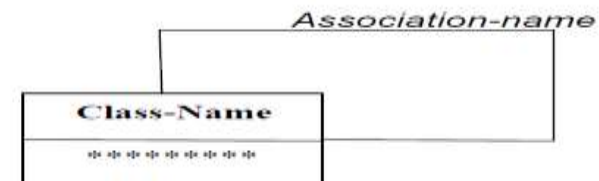
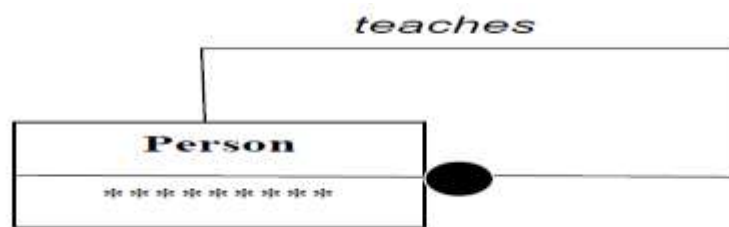
1. Unary,

2. Binary,

3. Ternary or n-ary

Unary Association

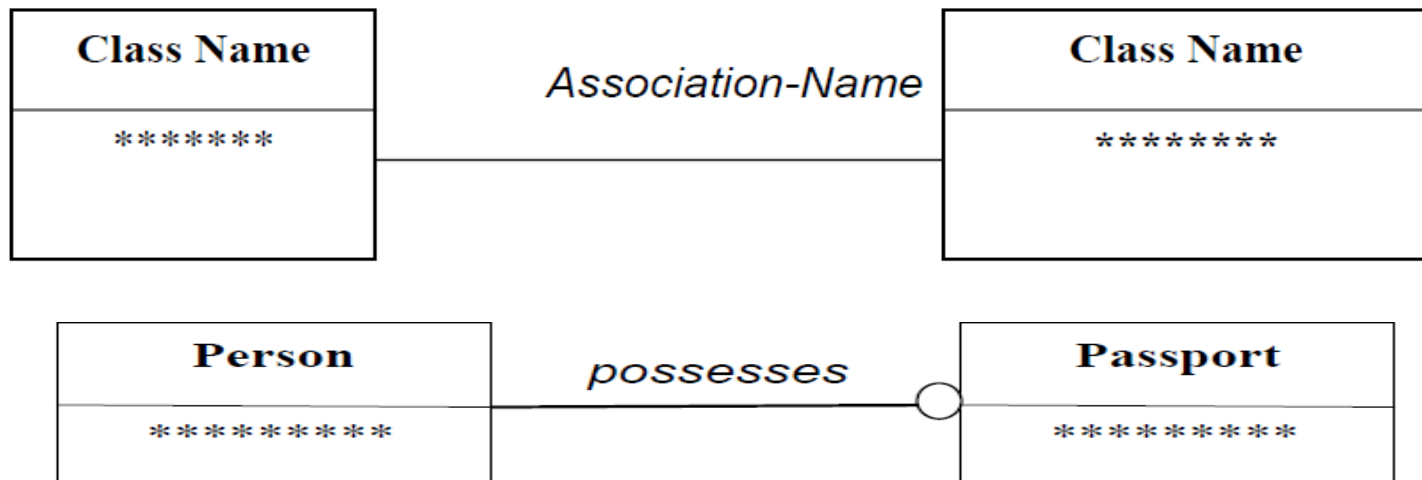
- Unary association is **between the same class**



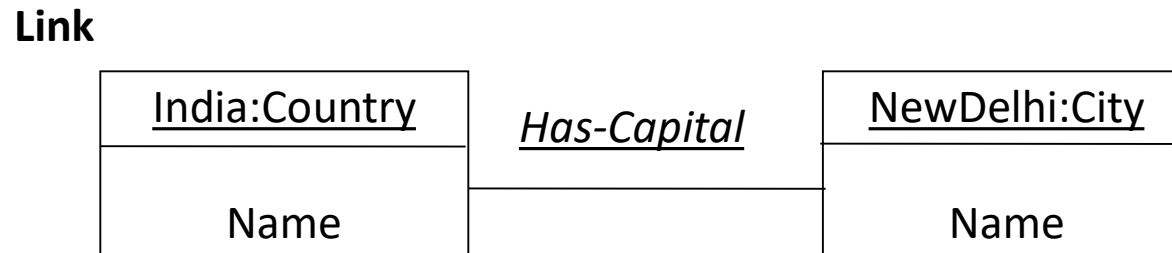
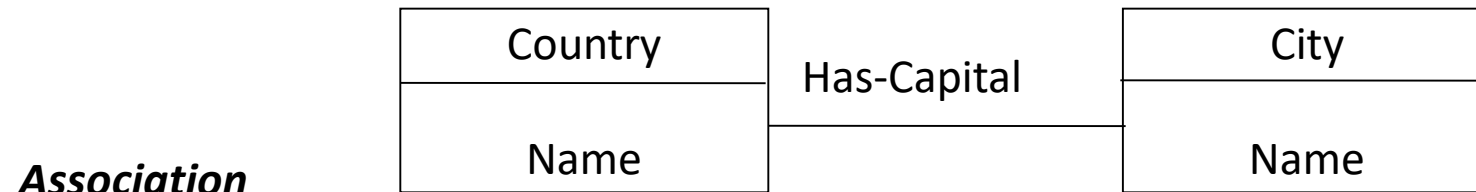
- Example of unary association is **Person teaches Person** as shown
- Other examples of unary association can be Person marries a Person, Person manages Person etc.

Binary Association

- Associations are generally **bidirectional** (traverse either direction)
- A binary association is an association **between two classes**



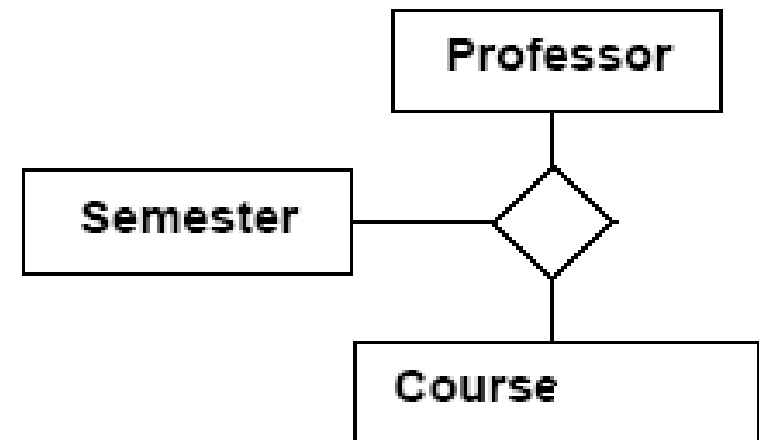
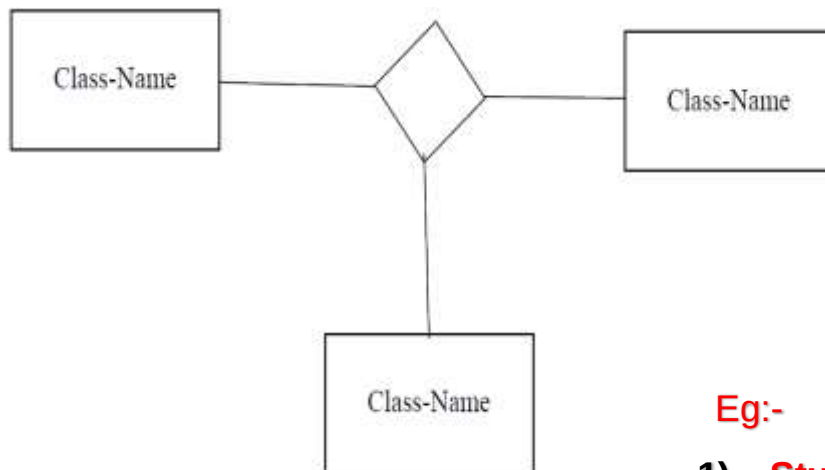
- Example of binary association is “Person possesses a Passport”.



Note: (i) **Link** is a relationship between **objects**
 (ii) **Association** is a relationship between **classes**

Ternary association

- Ternary association is an **atomic unit** and cannot be subdivided into binary associations without losing information
- The OMT symbol for ternary and n-ary associations is a diamond with lines connecting to related classes



Eg:-

1) **Student , Notebook, Pen**

2) **Hotel, Customer, Food**

Multiplicity

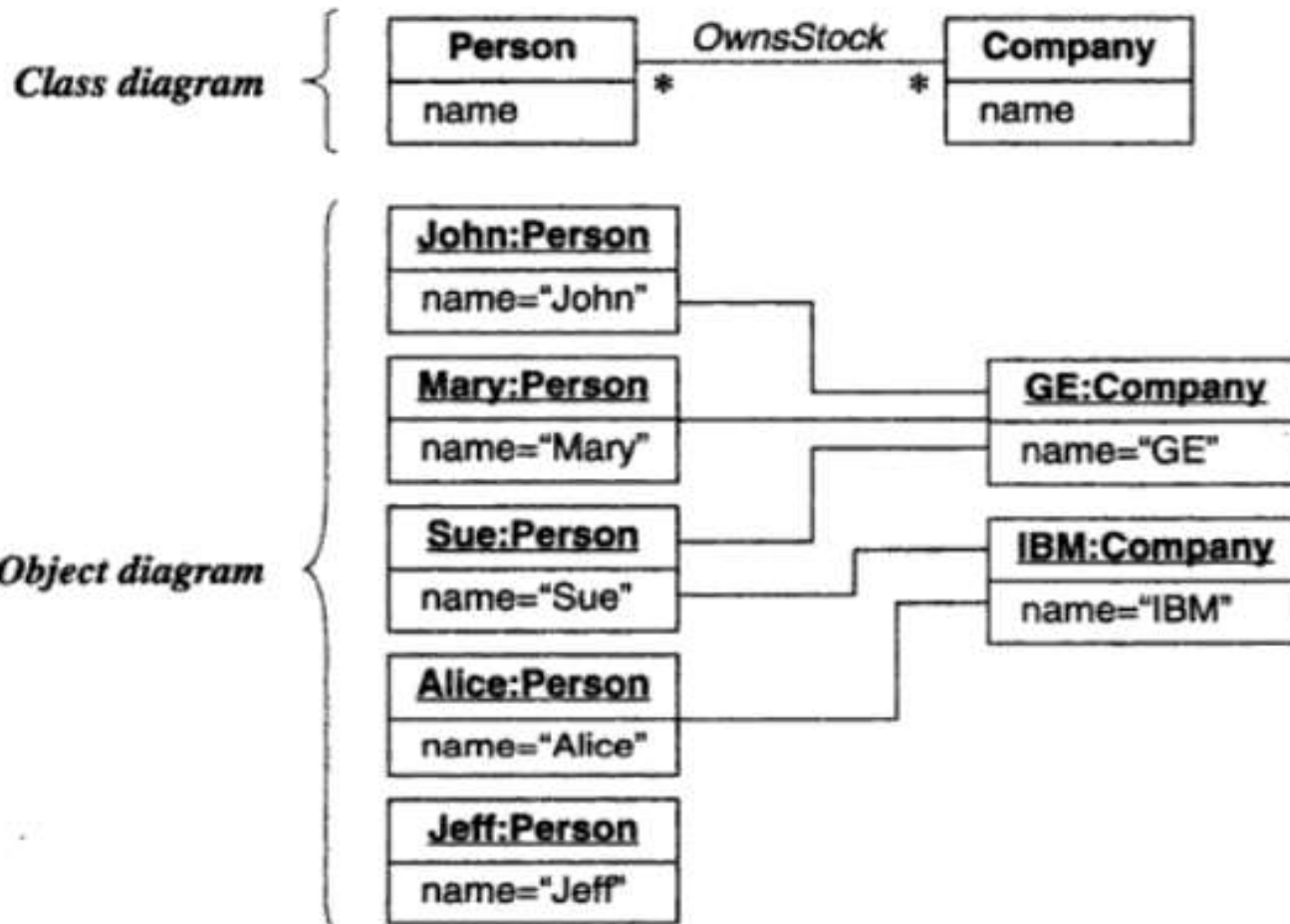
- Multiplicity specifies how many instances of one class may relate to a single instance of an associated class
- It constraints the number of related objects.
- It is often described as being one or more, but it is a subset of non-negative integers.

Multiplicity

- ✓ UML specifies a symbol * (asterisk) , denotes more (zero or more) .
- ✓ A line without multiplicity symbols indicates a **one-to-one association**
- ✓ In the general case, multiplicity can be specified with a number or set of intervals such as
 - “1” indicates exactly one
 - “1+” indicates one or more
 - “3-5” indicates three to five
 - “2,4,5” indicate two , four, five alone

Multiplicity

(many to many association)

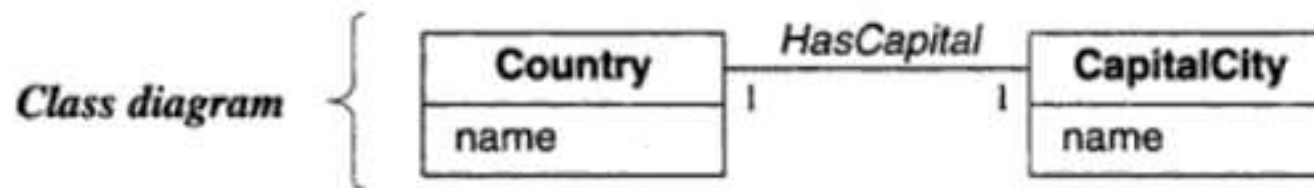


Multiplicity

- Associations may have different *cardinality* (*multiplicity*)
- **One-to-one**: one instance of the first class associated with one instance of the second class
- **One-to-many**: one instance of the first class associated with many instances of the second class
- **Many-to-many**: many instances of the first class associated with many instances of the second class

Multiplicity

(One to One association)



Zero or One multiplicity



Zero-or-one multiplicity. It may be optional whether an object is involved in an association.

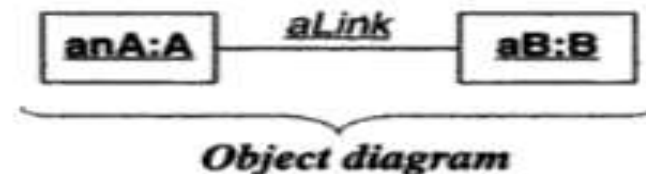
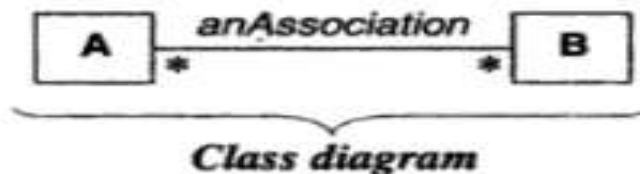


Figure 3.10 Association vs. link. A pair of objects can be instantiated at most once per association (except for bags and sequences).

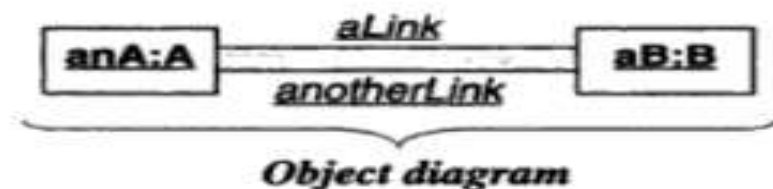
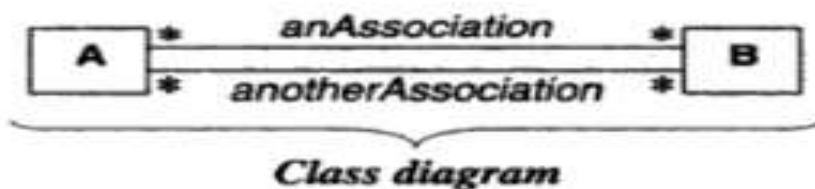


Figure 3.11 Association vs. link. You can use multiple associations to model multiple links between the same objects.

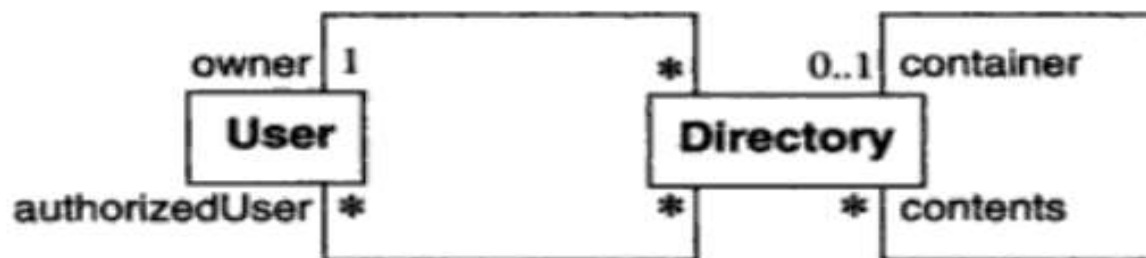
Association End Names



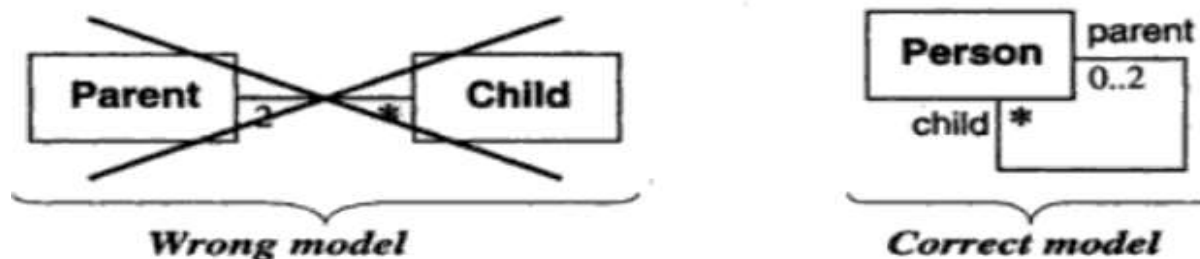
employee	employer
Joe Doe	Simplex
Mary Brown	Simplex
Jean Smith	United Widgets

Association end names. Each end of an association can have a name.

- Each directory has exactly one user who is an owner and many users who are authorized to use the directory.



- Association end names are necessary for associations between two objects of the same class



3.14 Association end names. Use association end names to model multiple references to the same class.

Ordering

- “many” association end have no explicit order

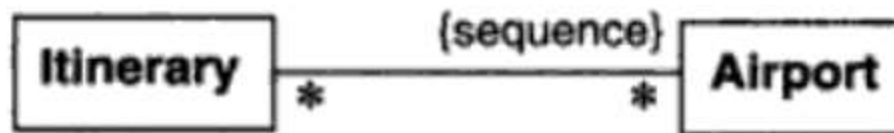


Ordering the objects for an association end. Ordering sometimes occurs for “many” multiplicity.

- Each ‘window’ on a ‘screen’ occurs at most once.
- Window has explicit order, only top most “window” is visible on “screen”

Bags and Sequences

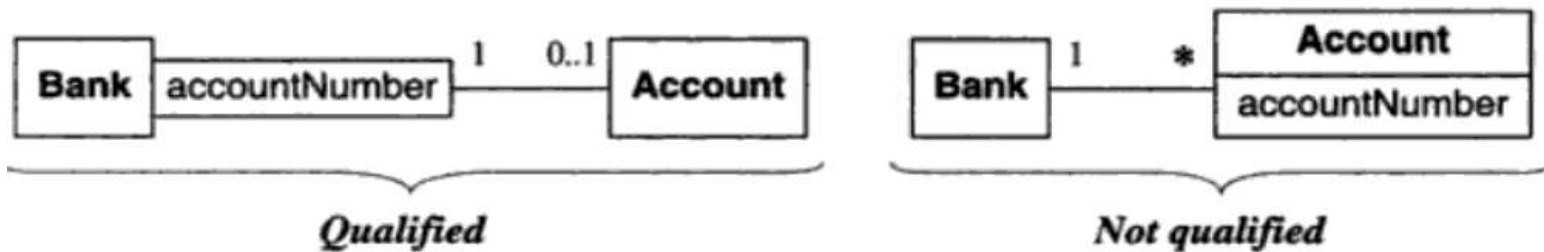
- A **bag** is a collection of elements with duplicates allowed
- A **sequence** is an ordered collection of elements with duplicates allowed



An example of a sequence. An itinerary may visit multiple airports, so you should use *{sequence}* and not *{ordered}*.

Qualified Association

- A qualified association is an association in which an attribute called **qualifier**.
- **Qualifier** disambiguates the objects for “many” association



- *A bank services multiple accounts, An a/c belongs to a single bank, within a context of a bank, the accountNumber specifies a unique account*
- **Bank** and **Account** are **classes**,
- **accountNumber** is the **qualifier**

Association Class

- You can describe links of an association using attributes and operations as like objects of class.
- An association class is an association that is also a class.

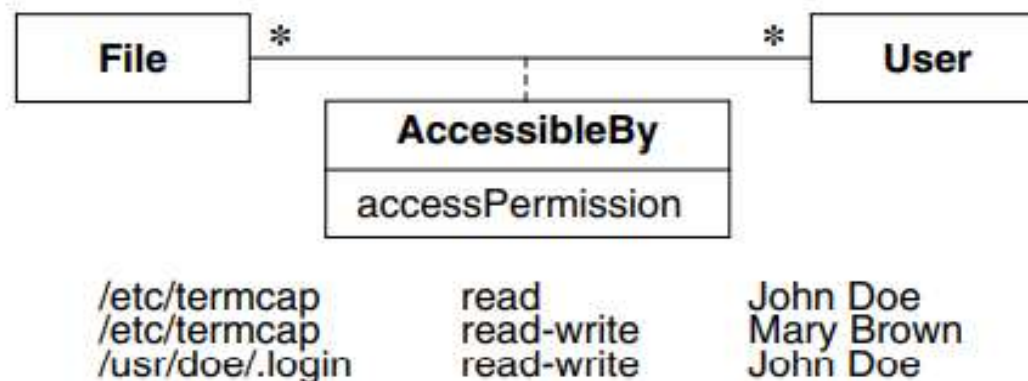


Figure 3.17 An association class. The links of an association can have attributes.

Many-to-many associations provide a compelling rationale for association classes. Attributes for such associations unmistakably belong to the link and cannot be ascribed to either object. In Figure 3.17, *accessPermission* is a joint property of *File* and *User* and cannot be attached to either *File* or *User* alone without losing information.

Association Class

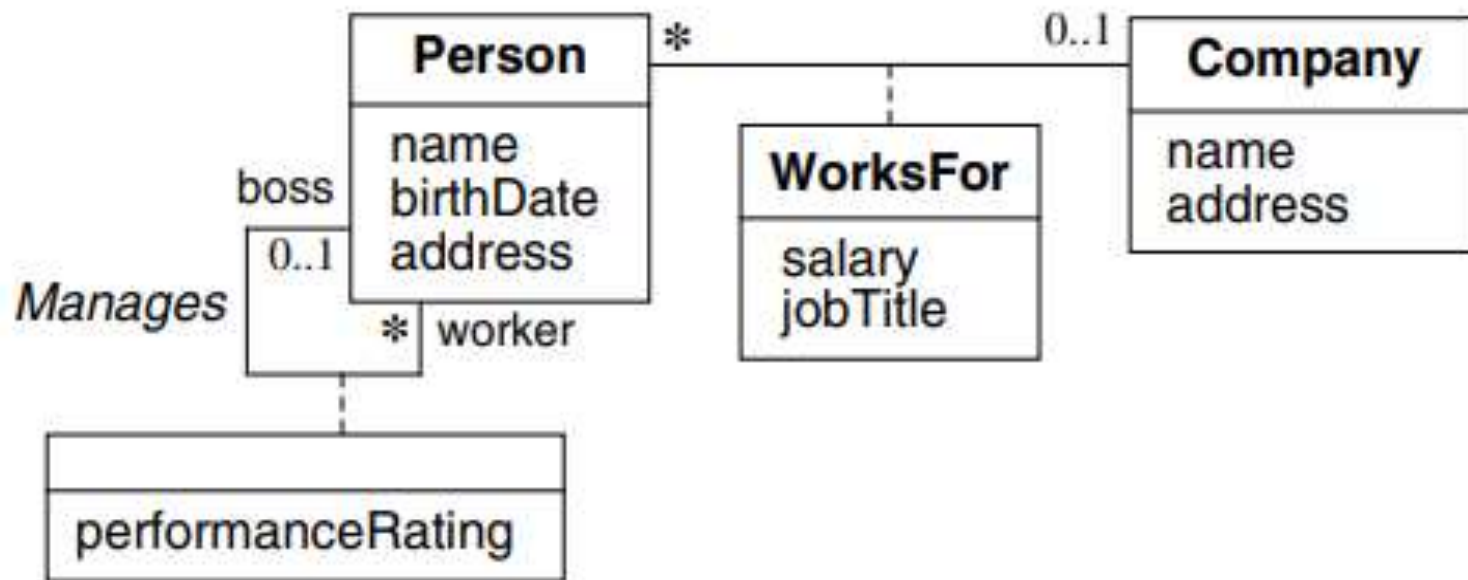


Figure 3.18 Association classes. Attributes may also occur for one-to-many and one-to-one associations.

Proper use of Association class

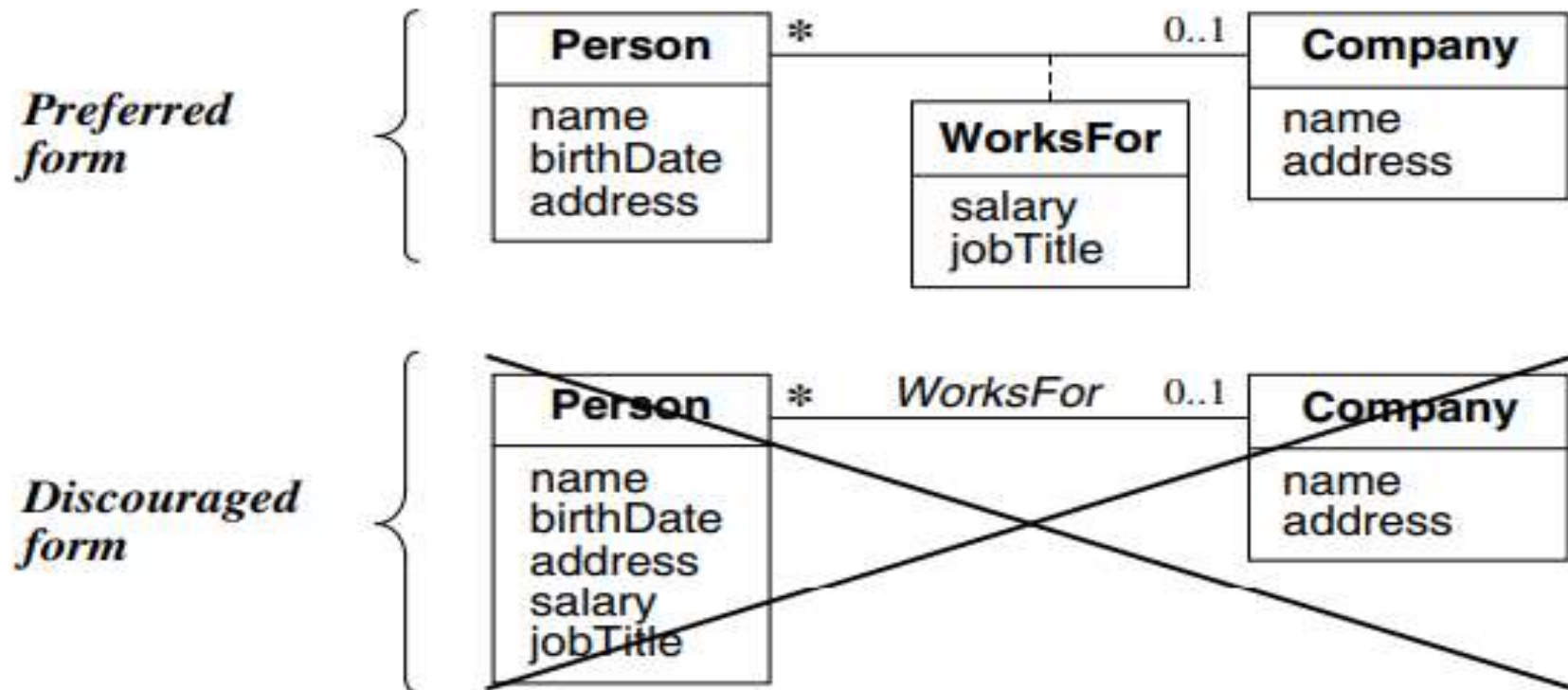


Figure 3.19 Proper use of association classes. Do not fold attributes of an association into a class.

1. I love U
2. a person orders same food many times, but maintain some order
3. Customer watching TV programs randomly
4. Execute program one by one – no repeat
5. Connecting two classes
6. Relationship between instances
7. wrapping up of data under a single unit.
8. It is an cannot be subdivided into binary association
9. Modeling purpose
10. It describes the life history of object
11. It is an adjective
12. Minimum one but not more than 3
13. Binary multiplicity
14. each Passenger will be allocated to exactly one Seat

**1 – d, 2 – a, 3 – f, 4 – c, 5 – h, 6 – i, 7 – m,
 8 – b, 9 – j, 10 – g, 11 – n, 12 – k, 13 – e,
 14 – l**

- a) Not bag and not ordered
- b) atomic unit
- c) Ordered
- d) Modeling Language missing but “Unified” exists
- e) 0..1
- f) It s a bag
- g) State model
- h) An association
- i) Link
- j) visualization
- k) 1..3
- l) No need to specify any multiplicity
- m) Encapsulation

What is grouping and How to?

Edible items	Electronics , Projector, Dog, Peacock
Vegetarian,	black-board , Monkey, piano, violin
Ice-cream,,	chalk-piece , flute
Choco, duster,	Maruthi-Suzuki,
Veg-pulav, hen,	
Non-vegetarian	
Macrun, fish, love-birds	

Three purposes to have Generalization:

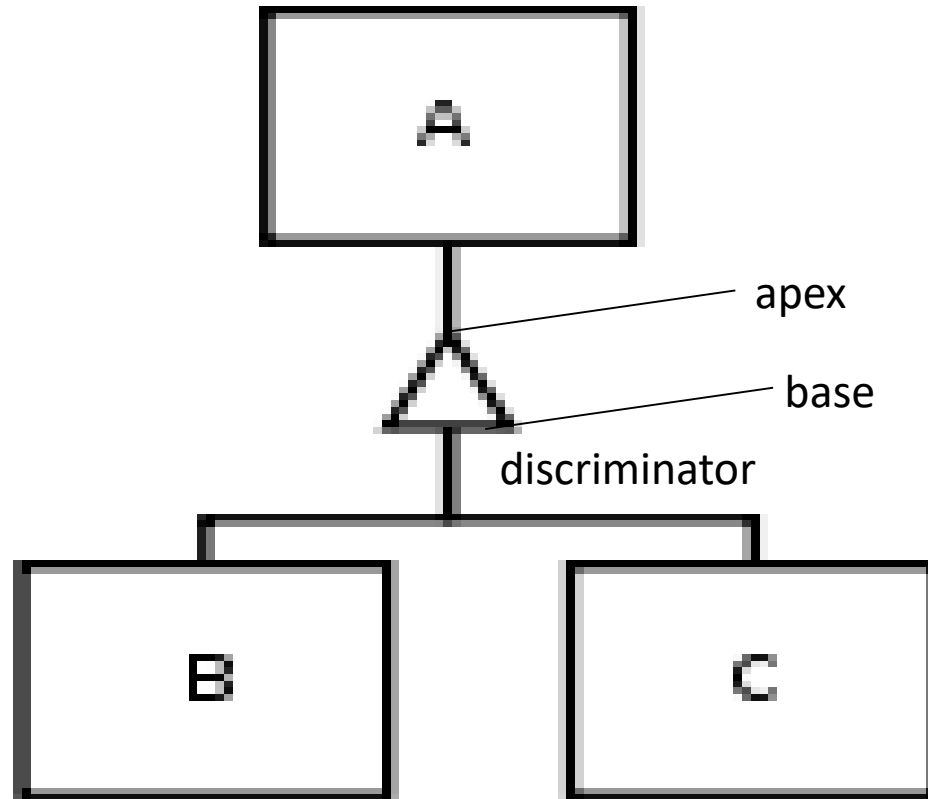
- **Support for Polymorphism**
- **To structure the description of objects** (you are forming taxonomy and organizing objects)
- **To enable reuse of code**

6.Generalization and Inheritance

- It allows for **sharing similarities among classes** while preserving their differences
- Generalization is the **relationship between a class and one or more refined versions** of it.
- The class being refined is called the **superclass**.
- Each refined version is called a **sub-class**
- Generalization is some-times called the **is-a** relationship
- The term **ancestor and descendent** refer to generalization of classes across multiple levels

6. Generalization and Inheritance

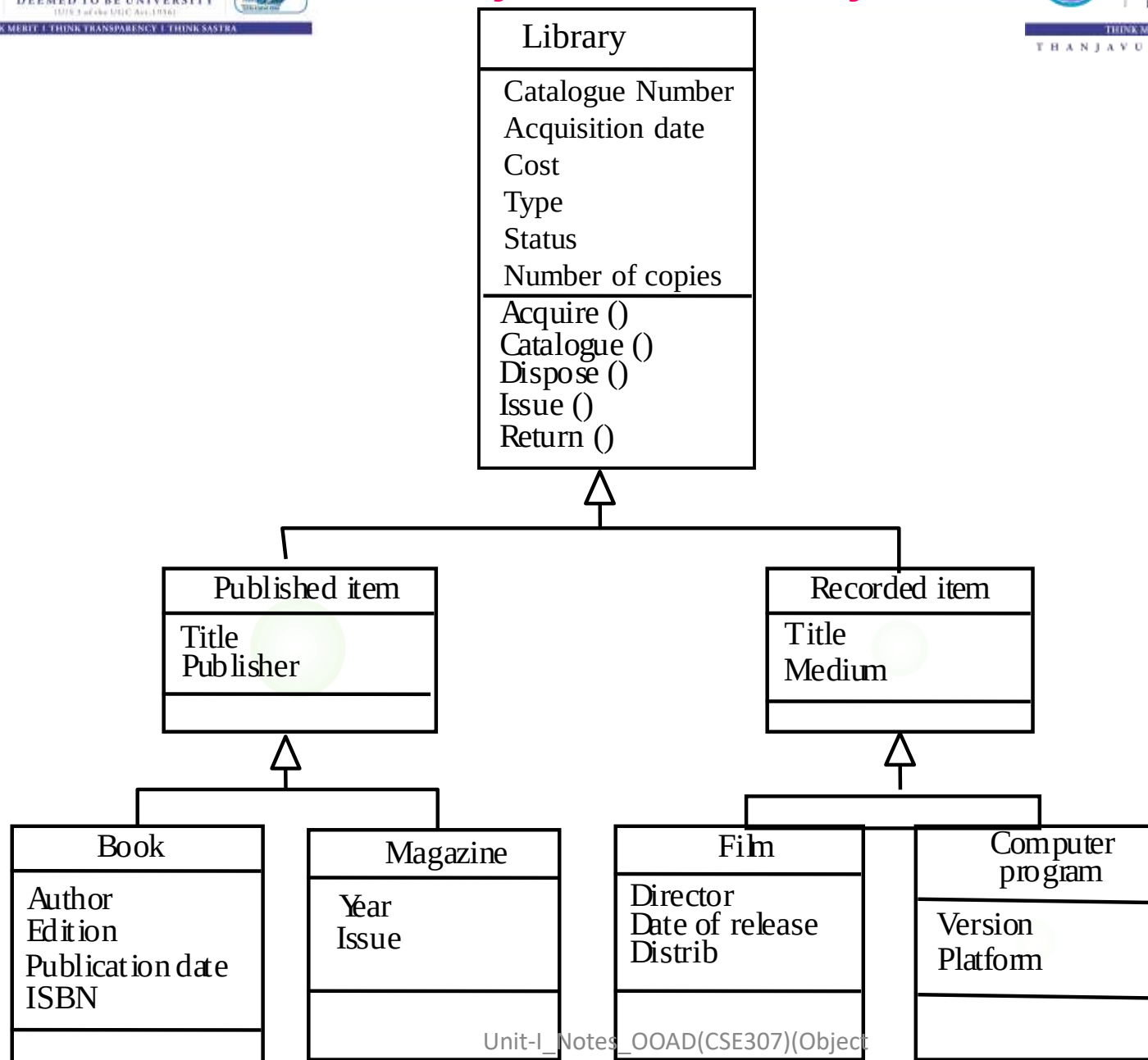
- The notation for generalization is a **triangle** connecting a superclass to its subclasses
- The **superclass** is connected by a line to the **apex(top)** of the triangle
- The **subclasses** are connected by lines to horizontal bar attached to the **base of the triangle**
- The words written next to the triangles are discriminators, it is an attribute of enumeration type.



Use of Generalization

- Useful for both **conceptual modeling and implementation**
- Facilitates modeling by structuring classes that capturing **what is similar and what is different about classes**
- Useful for **reusing of code**
- Note:
 - **Generalization**: superclass generalizes the subclasses or The superclass is a *generalization* of the subclass
 - **Specialization**: subclasses refine or specialize the superclass or The subclass is a *specialization* of the superclass

Library class hierarchy



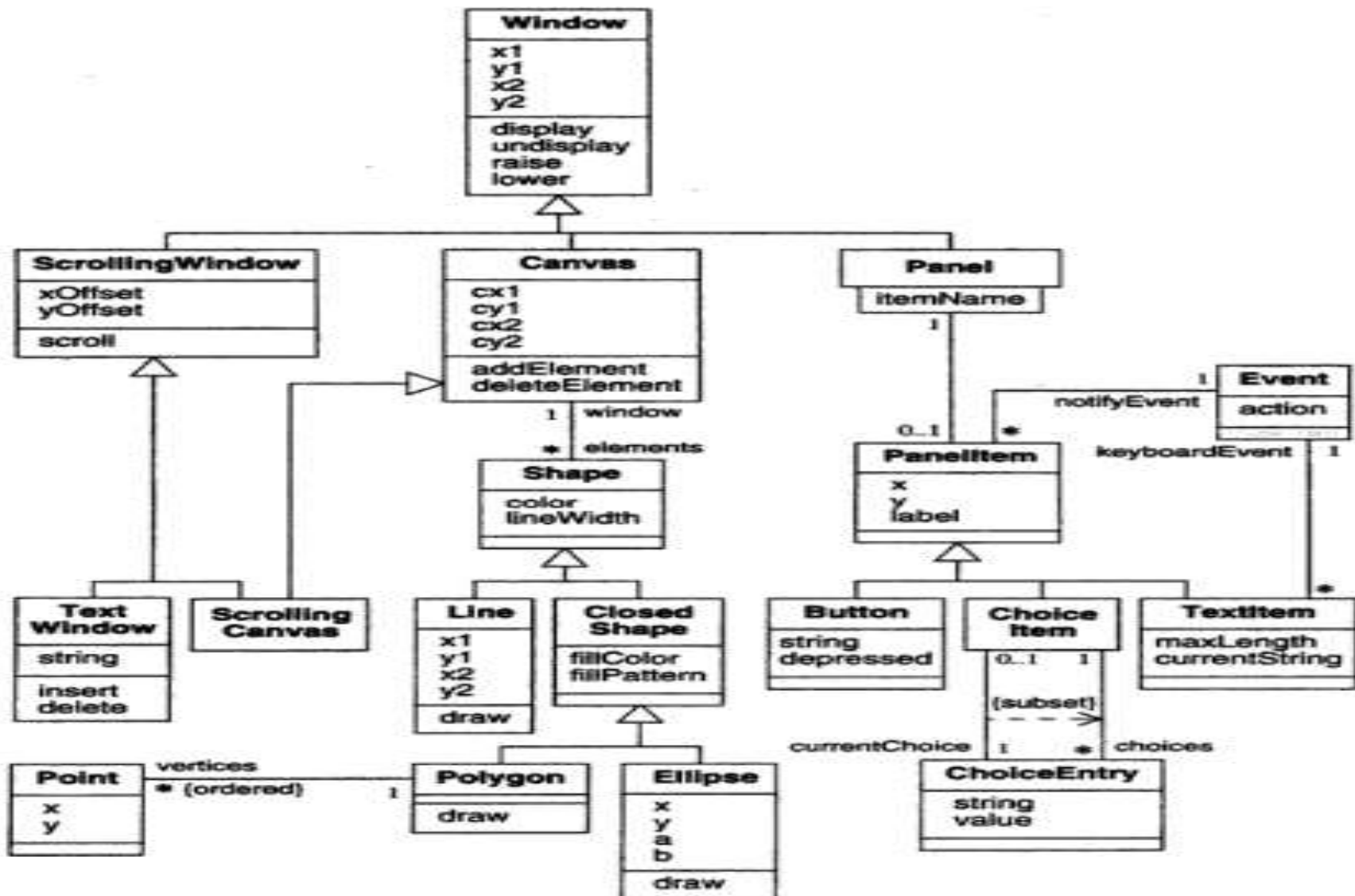
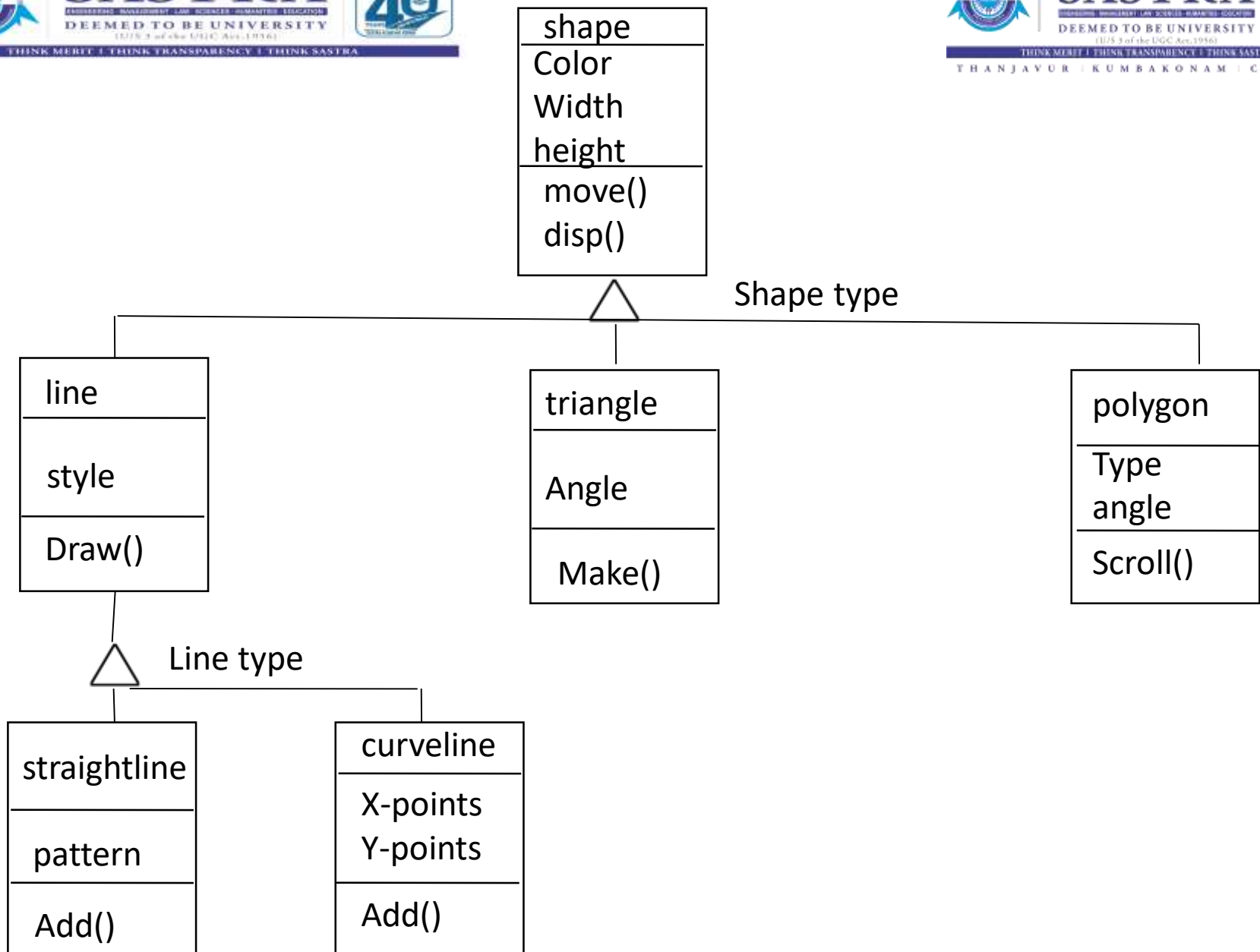
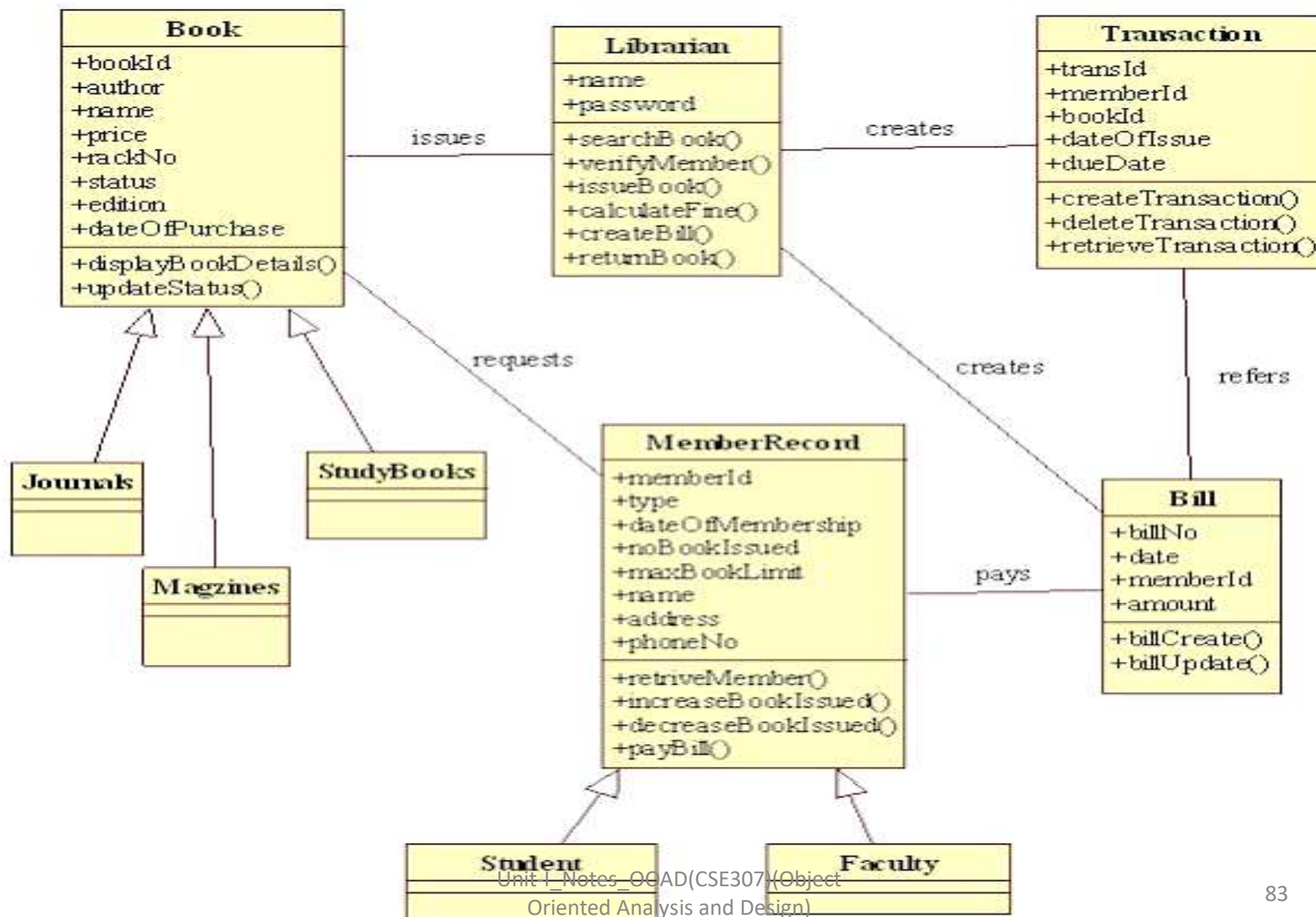


Figure 3.26 Class model of a windowing system





Navigation of a class model

- Navigation of class model is important because it exercise a model and **uncover hidden flaws and omissions** so that you can repair them. The navigation can be done by manually or writing navigation expressions
- Navigation expressions
 1. What transactions occurred for a credit card account within a time interval?
 2. What volume of transactions were handled by an institution in the last year?
 3. What customers patronized a merchant in the last year by any kind of credit card?
 4. How many credit card accounts does a customer currently have?

Problem with solution

(5) Figure E3.8 is a portion of a class diagram for a computer system for laying out a newspaper. The system handles newspaper pages which may contain, among other things, columns of text. The user may edit the width and length of a column of text, move it around on a page, or move it from one page to another. As shown, a column is displayed on exactly one page.

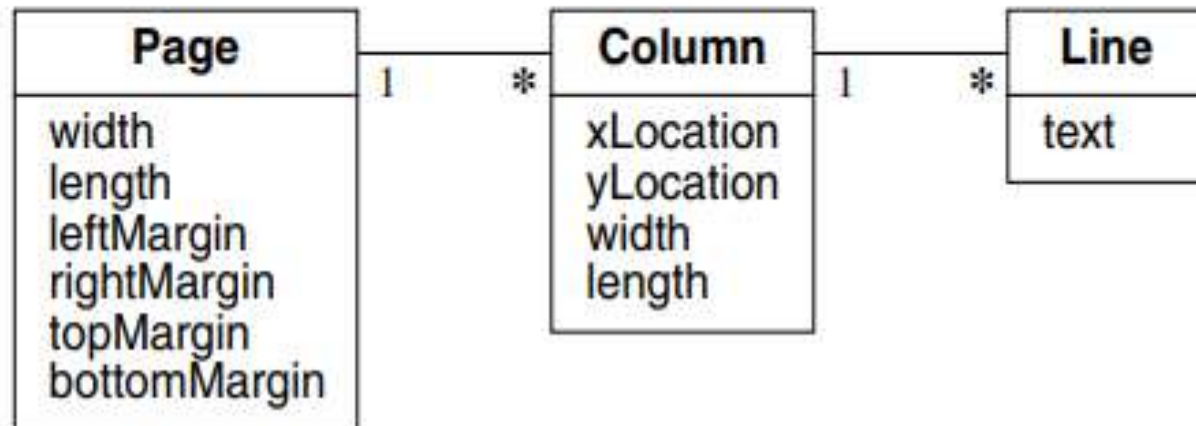


Figure E3.8 Portion of a class diagram for a newspaper publishing system

Advanced Class Modeling

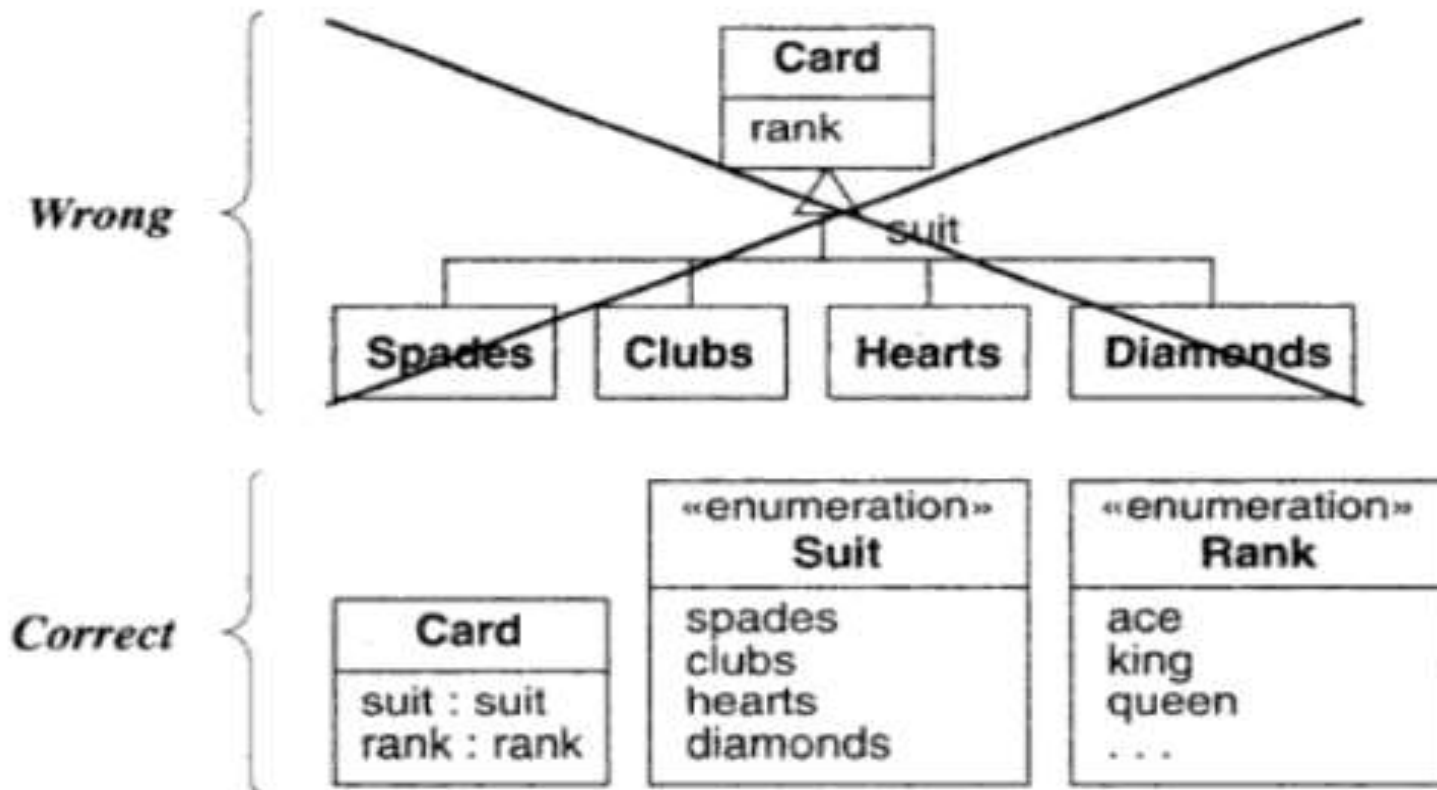
Unit-1 Part-2

Introduction

- Advanced class model provides advanced topics for improved class modeling. They are:
 1. Advanced object and class concepts
 2. Association end names
 3. N-ary associations
 4. Aggregation
 5. Abstract classes
 6. Multiple inheritance
 7. Metadata
 8. Reification
 9. Constraints
 10. Candidate keys

• Enumerations

- is a data type, has a finite set of values
- Use key word ***enumeration*** <<enumeration>>

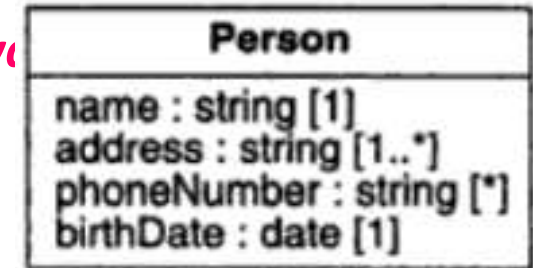


Enumeration



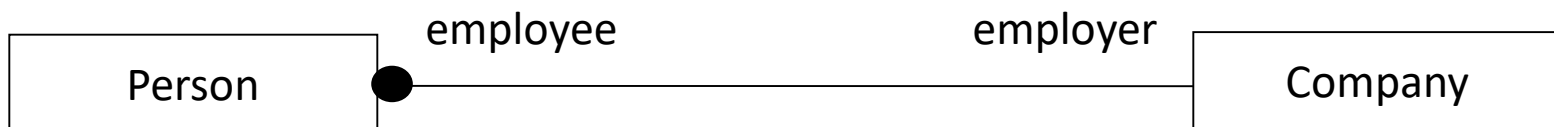
Figure 4.1 Examples of enumerations. Enumerations often occur and are important to users. Implementations must enforce the finite set of values.

- **Multiplicity for attributes**
 - *Specify whether an attribute is single or multiple, mandatory or optional*
 - *Single value [1]*
 - *Optional single value [0..1]*
 - *Many [*]*
 - *If not specified, an attribute is single value [1]*
- **Scope**
 - *Indicates if a feature applies to an object or a class*
- **Visibility**
 - *Public*
 - *Private*
 - *Protected*
 - *package*



2. Association Ends

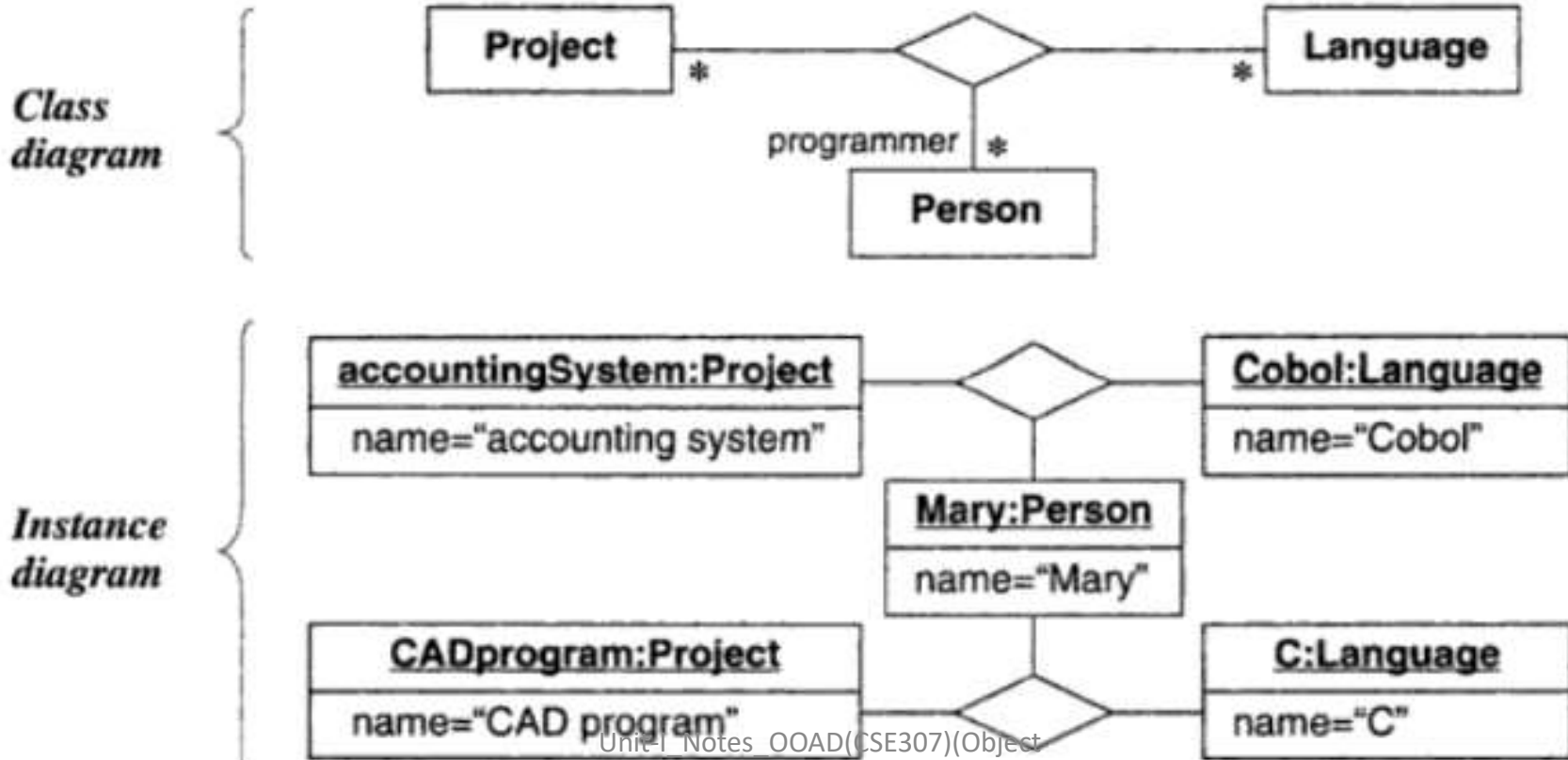
- An **association end** is an end of an association.
- **Association end name**
- **Multiplicity**
- **Ordering**
- **Bags and sequences**
- **Qualification** – one or more qualifier to disambiguate objects
- **Aggregation** – association end may be an **aggregate/constituent**
- **Changeability**
- **Navigability** – an association may be traversed either direction.
- **Visibility** - private, public, protected or package.



Works-for

3.N-ary association

- Ternary association is an atomic unit and **cannot be subdivided into binary associations without losing information**
- The OMT symbol for ternary and **n-ary** associations is a diamond with lines connecting to related classes



- A professor teaches a listed course during a semester.
- The resulting delivered course has a room number and any number of textbooks.

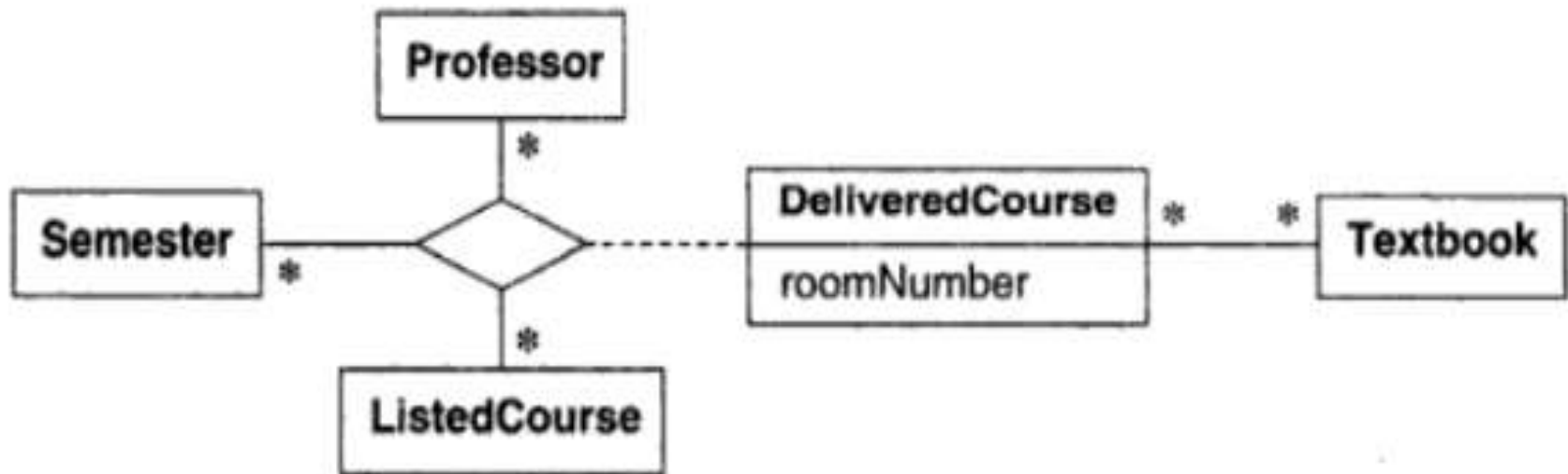
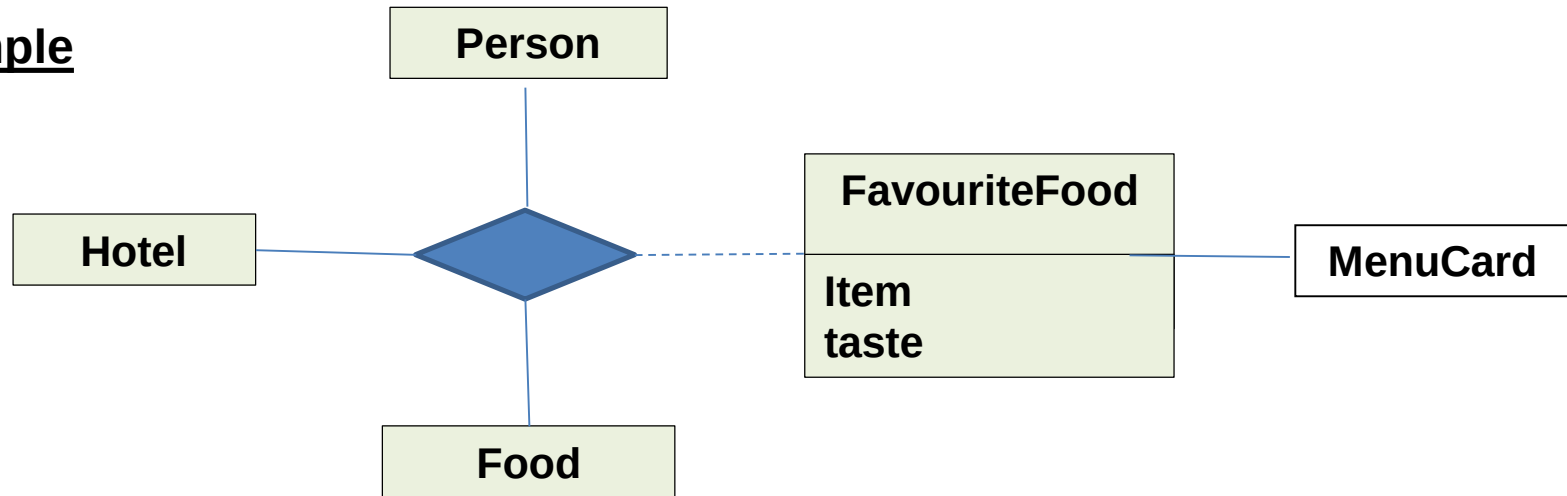


Figure 4.7 Another ternary association. N-ary associations are full-fledged associations and can have association classes.

Example

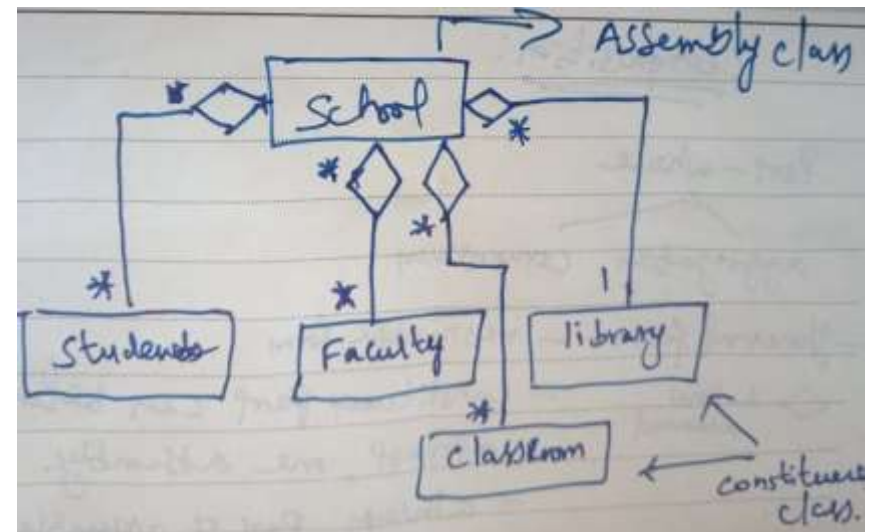
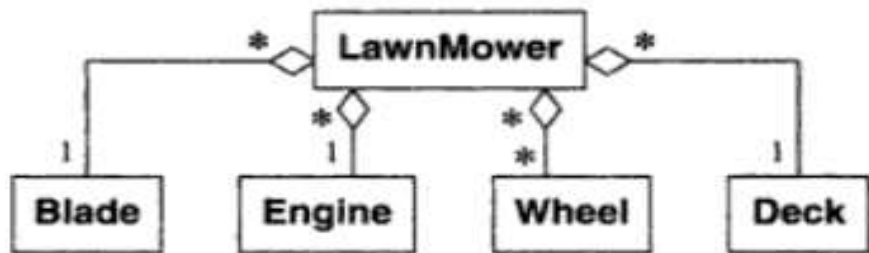


a person eats a food from a hotel. FavouriteFood item can be chosen from MenuCard

N-ary association with association classes

4. Aggregation

- Aggregation is a strong form of association in which an aggregation object is made of constituent parts.
- constituents are part of the aggregate
- Aggregate is extended object



- Aggregation is Transitivity
 - A is part of B, B is part of C; then A is part of C
- Aggregation is also anti-symmetric
 - A is part of B; B is not part of A

Aggregation Vs Association

Aggregation

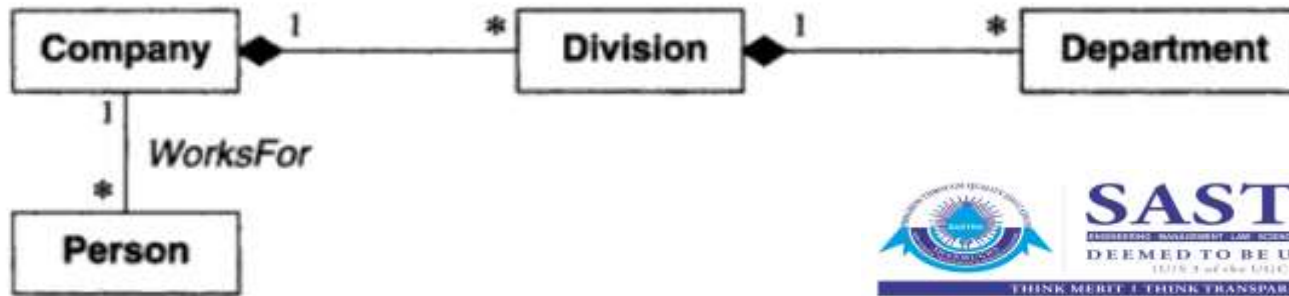
- Dependent concept.
- Tightly-bound form of relationship.
- One object is subordinate to the other.
- Some operations and attributes automatically applied from whole to part object classes.

Association

- Independent concept.
- Loosely-bound form of relationship.
- One object class is not subordinate to the other.
- Operations and attributes are unique with independent classes.

Aggregation Vs. Composition

- Two forms of Part-Whole : *aggregation, composition*
- Composition – more restrictive - A constituent part can belong to at most one assembly*



- Propagation of Operations: is the automatic application of an operation to a network of objects

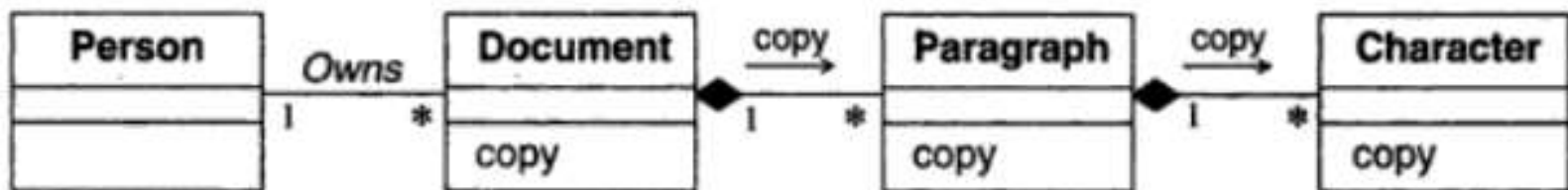
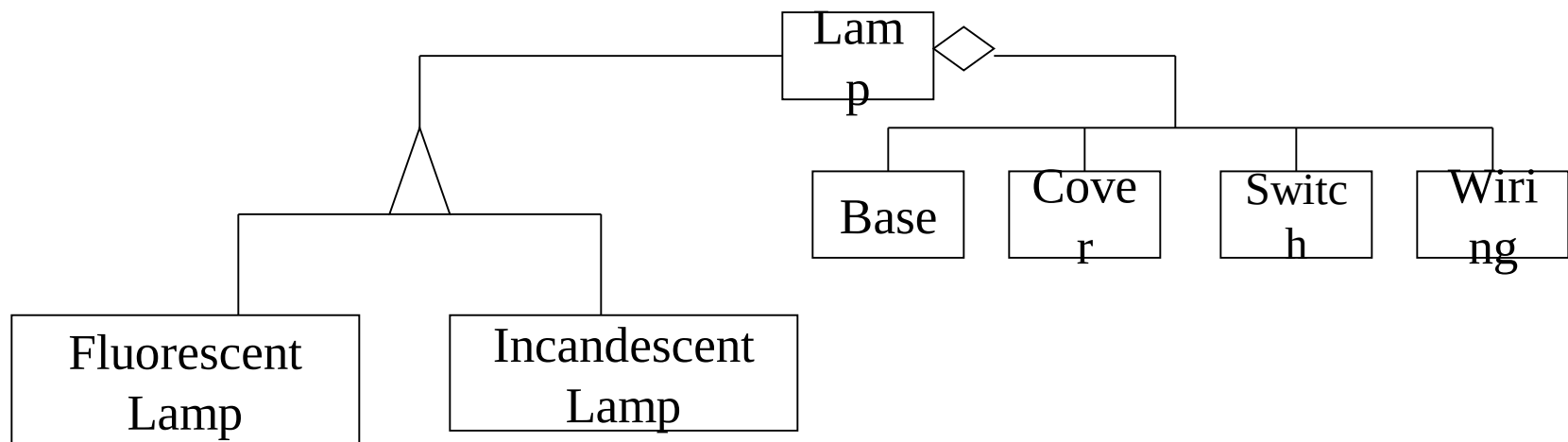


Figure 4.11 Propagation. You can propagate operations across aggregations and compositions.

Aggregation Vs Generalization

- Aggregation relates **instances**.
- Aggregation is often called “**a-part-of**” relationship.
- “And” relationship between object classes.
- An aggregation tree is composed of object instances.

- Generalization relates **classes**.
- Generalization is often called “**a-kind-of**” or “**is-a**” relationship.
- “Or” relationship between object classes.
- A generalization tree is composed of object classes.



- An abstract is a class that **has no direct instances** but whose descendent Classes have direct instances.
- A **Concrete** class is a class that is instantiable.
- Abstract classes may contain Abstract methods. Subclasses extending an abstract class may implement these (inherited) Abstract methods.
 - If the child class implements all such Abstract methods, it becomes a concrete class.
 - if it does not, the child class also becomes an Abstract class.

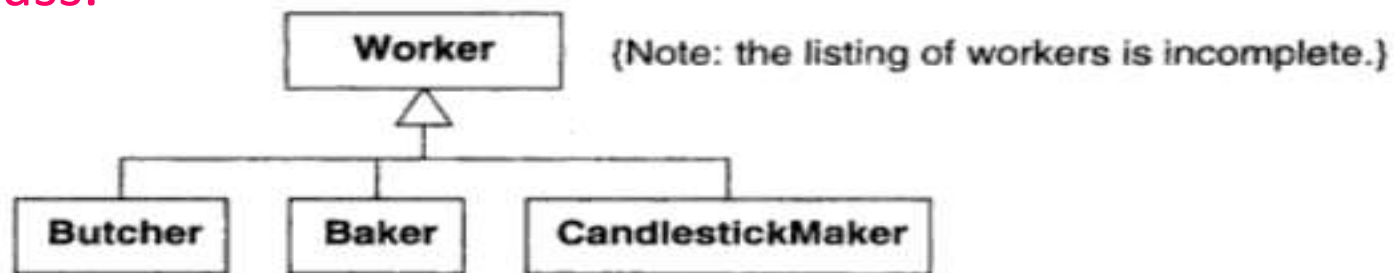


Figure 4.12 Concrete classes. A concrete class is instantiable; that is, it can have direct instances.

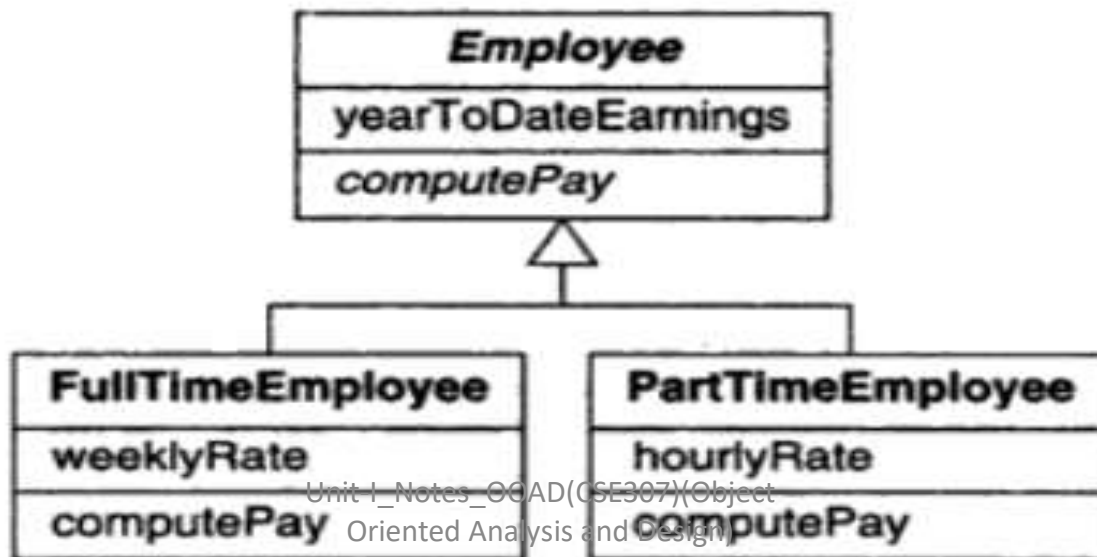
Abstract Vs. Concrete classes

Abstract classes

- Organize features to several classes.
- Classes have no implementation.
- Cannot have direct instances
- May contain abstract methods

Concrete(conceptual) classes

- Organize features to single class.
- Classes have full implementation.
- Can have direct instances
- Can not have abstract methods.



6. Multiple inheritance

- Multiple inheritance permits **a class to have more than one super class and to inherit features from all parents**. This permits mixing of information from two or more classes
- A class with more than one super class is called a join class
- Advantages :
 - Greater power in specifying classes
 - Increased opportunity for reuse
- Disadvantage :
 - Loss of implementation simplicity

Multiple inheritance Example

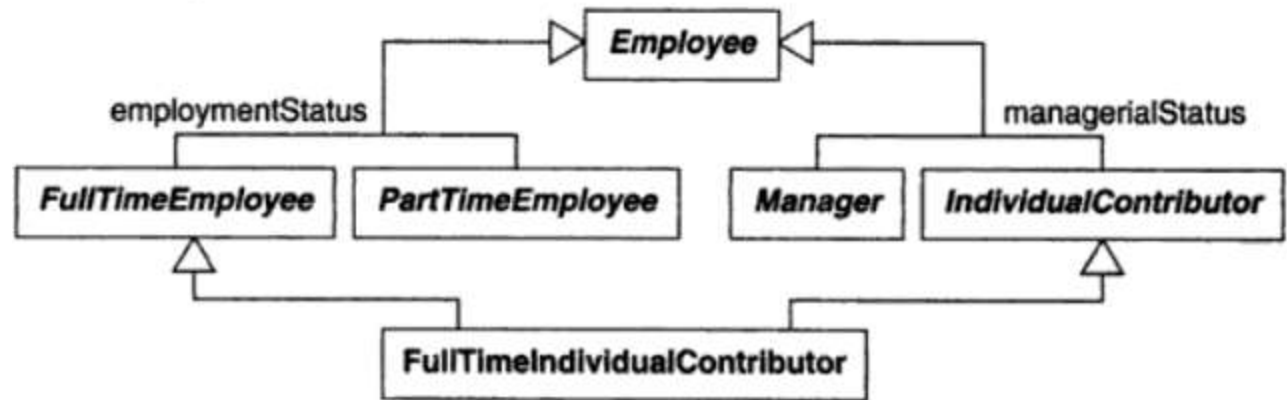
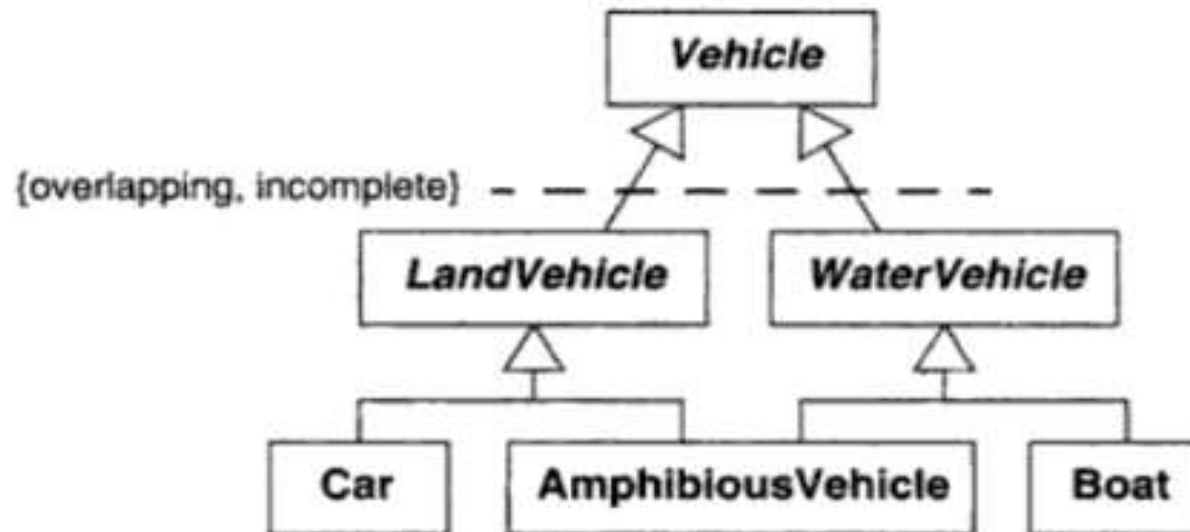


Figure 4.15 Multiple inheritance from disjoint classes. This is the most common form of multiple inheritance.



16 Multiple inheritance from overlapping classes. This form of

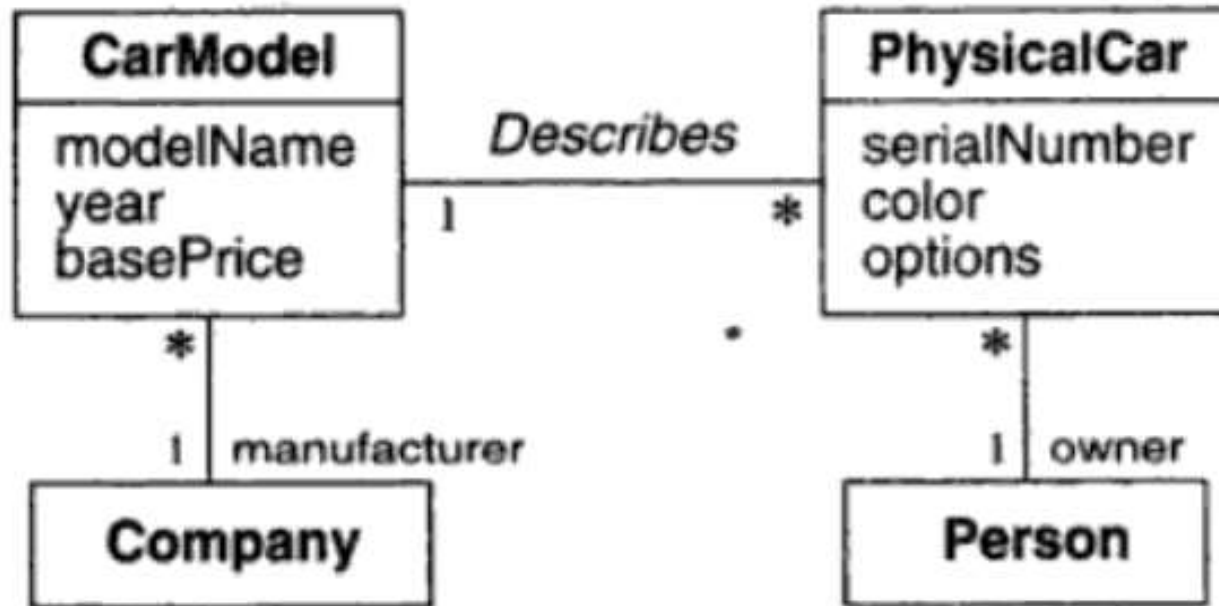
Delegation

- **Delegation** means that you use an object of another class as an instance variable, and forward messages to the instance.
- **Delegation** can be viewed as a relationship between objects where one object **forwards** certain method calls to another object, called its **delegate**
- **An Object forwards an operation to another object for execution.**

7. Metadata

- Metadata is data that **describes** other data
- For example:
 - Definition of a class is metadata
 - Real-world applications have metadata
 - Part catalog describes manufactured parts
 - Blue print describes a house
 - Dictionaries describe a word

Metadata



- CarModel is **Metadata** describes PhysicalCar which is **Data**
- A person (xyz) holds a car (IAX102, blue, petrol) its description (ertiga, 2021, 10Lk) from Maruthi

8.Reification

- Reification is the promotion of something that is not an object into an object.
- It is a helpful technique for meta applications
- It is useful to promote
 - Attributes
 - Methods
 - Constraints
 - Control information
- Example
 - A database manager provides a general-purpose solution to accessing data reliably and quickly for multiple users

Reification - Example

Figure 4.22 promotes the *substanceName* attribute to a class to capture the many-to-many relationship between *Substance* and *SubstanceName*. A chemical substance may have multiple aliases. For example, propylene may be referred to as *propylene* and C_3H_6 . Also, an alias may pertain to multiple chemical substances. Various mixtures of ethylene glycol and automotive additives may have the alias of *antifreeze*.

Reification:
*Promote attribute
to a class*

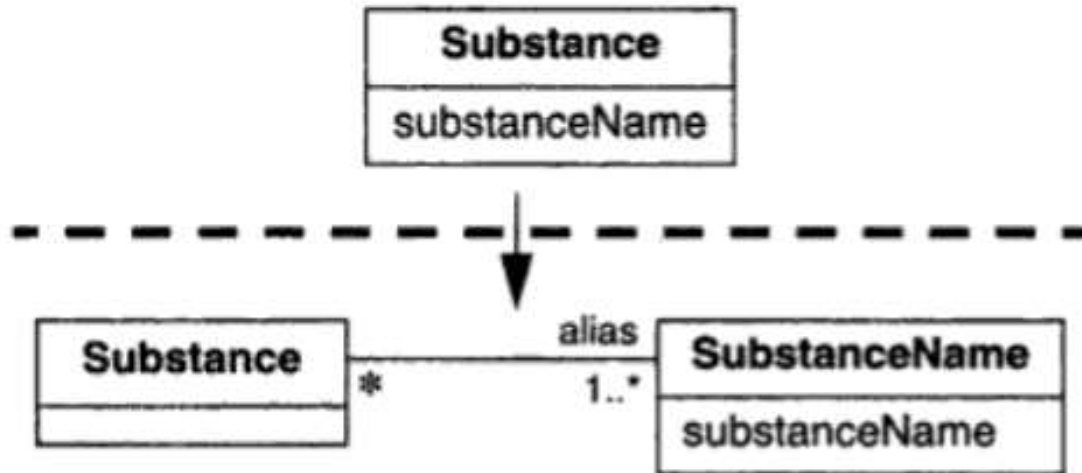


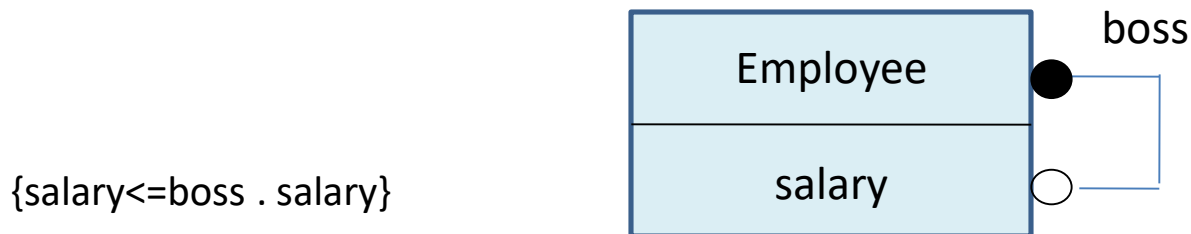
Figure 4.22 Reification. Reification is the promotion of something that is not an object into an object and can be helpful for meta applications.

9. Constraints

- Constraints are functional relationships between entities of an object model
- The term entity includes objects , classes , attributes , links, and associations
- The constraint restricts the values that entities can assume

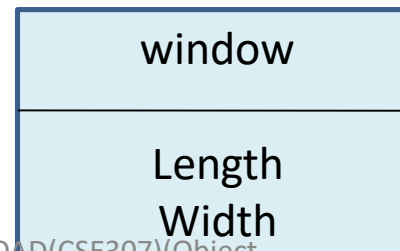
Constraints on objects

- No employee's salary can exceed the salary of the employee's boss



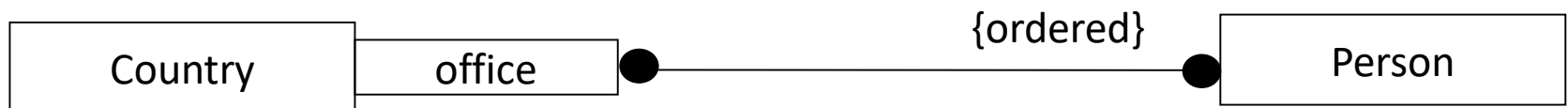
- No window will have an aspect ratio (length / width) of less than 0.8 or greater than 1.5

{0.8 ≤ length/width ≤ 1.5}



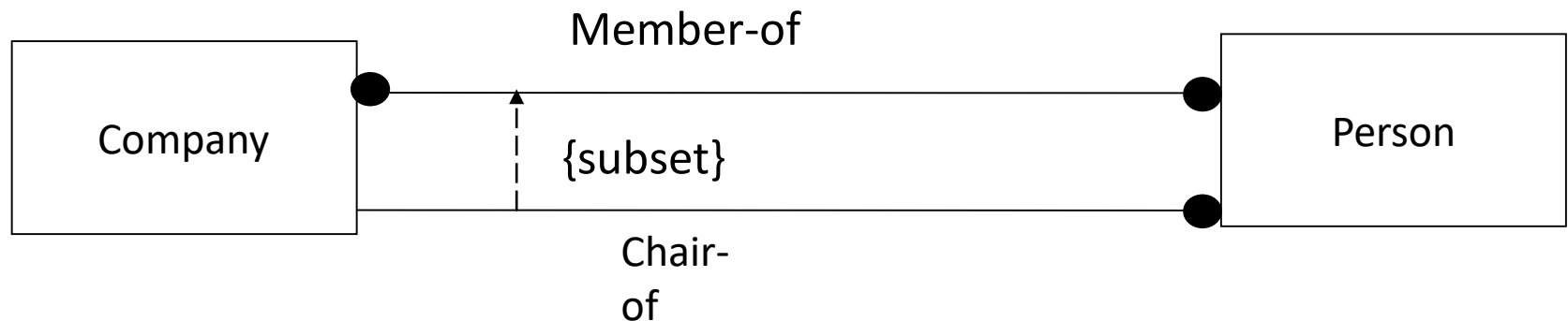
Constraints on links

- Multiplicity constrains an association. It restricts the number of objects related to a given object
- The notation “{ordered}” indicates that the element of the ‘many’ end of an association have an explicit order that must be preserved
- For example each office for each country has a set of persons who have held the office, ordered chronologically (such as president, chief justice, king)



General Constraints

- When an **association becomes a subset of another** there is a general constraint. We should **draw a dotted line between classes involved in the constraint** and specify the details with comments in braces.



10. Derived Data

- It is a function of one or more elements, which in turn may be derived
- Classes, associations, and attributes may be derived
- The **notation** for a derived element is a **slash** in front of the element name
- The **constraint** that determines the derivation should be noted near the class.

Derived Data - Example

Person
birthdate /age

CurrentDate

{age = CurrentDate - birthdate}

Packages

- Small and medium sized problems can fit a class model on a single page, however it is often difficult to grasp the entirety of a large model.
- A package is a group of elements (classes, associations, generalizations, and lesser packages) with a common theme.
- A package partitions a model, making it easier to understand and manage.
- Larger applications may require several tiers of packages
- A tabbed folder is the notation for a package
- Themes for forming packages
 - Dominant classes
 - Dominant relationships
 - Major aspects of functionality, and
 - symmetry

Packages – example

